

KINEMA NEXUS

Explicación del código

Aaron Poveda Joao

print_pose().....	26
print_all().....	27
get_arm_landmarks().....	28
El return.....	29
pose_math.py.....	31
pose_math.py explicación.....	33
Importación de math.....	34
Vectores	34
normalize_vector()	36
Cálculo del módulo.....	37
Normalización	38
Ángulos.....	38
Producto escalar.....	38
Módulos de los vectores	39
Evitar vectores de longitud cero	40
Cálculo del coseno.....	40
Protección frente a errores numéricos	40
Obtención del ángulo	41
Distancias	42
Resumen de pose_math.py	43
Pose_drawer.py	44
Pose_drawer.py explicación.....	53
Importación de librerías	53
draw_landmark().....	54
cv2.circle()	54
draw_connection()	55
cv2.line()	56
draw_angles().....	56
Cálculo del ángulo del brazo izquierdo	56
Cálculo del ángulo	57
Mostrar el ángulo sobre la imagen	58
Cálculo del brazo derecho	58
Bloque de distancias.....	59
draw_pose()	60
Copia de la imagen	60
Dibujar los hombros	60

Dibujar los codos	61
Dibujar las muñecas	61
Dibujar las conexiones.....	62
Dibujar los ángulos	62
Distancias desactivadas.....	63
Mostrar el resultado.....	63
Calibration.py	65
Calibration.py explicación	90
IMPORTACIONES	90
CONFIGURACIÓN.....	92
IMAGEN DE REFERENCIA.....	93
DETECTAR CUERPO SUPERIOR	96
CREAR HUMAN POSE	97
COMPROBAR ÁNGULOS	98
DIBUJAR LANDMARKS	100
MENSAJES.....	101
BUCLE PRINCIPAL DE CALIBRACIÓN	103
PREPARAR EL PANEL DE CÁMARA.....	108
MAIN	109
RELACIÓN CON LOS ARCHIVOS ANTERIORES.....	110
Relative_pose.py	113
Relative_pose.py Explicación	115
Importación de la función matemática.....	115
Posiciones relativas del brazo	115
Función calculate_relative_arm_pose	116
Documentación de la función	116
Brazo izquierdo.....	117
Vector del brazo derecho.....	120
Devolver vectores relativos.....	121
¿Por qué calculamos posiciones relativas?	123
Relación con la capa matemática.....	124
Importancia dentro del proyecto	125
Limits.py	126
Limits.py Explicación	126
Límites de movimiento.....	126
Límite del codo izquierdo.....	127

Relación con PoseMath.....	129
Estado actual de los límites.....	130
Robot_mapping.py.....	132
robot_mapping.py explicación.....	136
Importación de librerías.....	136
Configuración.....	136
Medidas provisionales del robot.....	137
Longitud del brazo del robot.....	137
Longitud del antebrazo del robot.....	138
Cargar calibración.....	138
Comprobar que existe el archivo.....	139
Abrir el archivo de calibración.....	139
Leer los datos JSON.....	140
Calcular escala.....	140
Comprobar la longitud humana.....	140
Calcular la escala.....	141
Crear mapeo humano-robot.....	141
Obtener las medidas del operador.....	141
Calcular escala del brazo izquierdo.....	142
Calcular escala del brazo derecho.....	143
Calcular escala del antebrazo derecho.....	143
Devolver configuración de mapeo.....	144
Cargar y crear el mapeo.....	144
Crear el mapeo.....	145
Flujo completo del archivo.....	145
Relación con Relative Pose.....	146
Relación con Robot Motion.....	147
Estado actual del mapeo.....	148
Robot_motion.py.....	149
robot_motion.py explicación.....	154
Importación de librerías.....	154
Límites provisionales del robot.....	155
Función clamp.....	156
Calcular movimiento del robot.....	157
Obtener vectores del brazo derecho.....	158
Ángulo entre brazo y antebrazo.....	160

Conversión provisional a articulaciones del robot.....	161
Aplicar límites.....	162
Crear configuración.....	163
Guardar JSON.....	164
Guardar la configuración.....	164
Función principal.....	165
Guardar el movimiento.....	166
Relación con los archivos anteriores.....	167
human_pose_detection.py.....	170
human_pose_detection.py Explicación.....	175
Importación de librerías y módulos.....	175
OpenCV.....	175
YOLO.....	175
draw_pose.....	176
calculate_relative_arm_pose.....	176
Cargar el mapeo del robot.....	177
Cargar modelo.....	177
Inicializar la cámara.....	177
Bucle principal.....	178
Capturar un frame.....	179
Comprobar captura.....	179
Inferencia.....	179
Crear objeto Pose.....	179
Obtener keypoints.....	180
Comprobar que existen keypoints.....	180
Obtener coordenadas y confianza.....	180
Mostrar la pose.....	185
Calcular pose relativa.....	185
Mostrar pose relativa.....	186
Calcular movimiento del robot.....	186
Mostrar movimiento del robot.....	187
Dibujar la pose.....	187
Liberar la cámara.....	187
Cerrar ventanas.....	188
Integración entre los diferentes módulos.....	189
Papel de Human Pose Detection dentro del proyecto.....	190

Estado actual de la integración	192
---------------------------------------	-----

Documentación del código — Kinema Nexus

Este documento tiene como objetivo **explicar el funcionamiento del código de Kinema Nexus de forma clara y progresiva**, de manera que cualquier persona que se incorpore al proyecto pueda entenderlo, independientemente de su nivel de programación.

La documentación comienza desde las estructuras y funciones más básicas y avanza progresivamente hacia las partes más complejas del sistema. Se explicará qué hace cada función, qué información recibe, qué operaciones realiza y qué resultado produce, incluyendo los conceptos matemáticos o las funciones de librerías externas necesarios para comprender su funcionamiento.

El objetivo no es enseñar programación desde cero, sino **permitir que una persona que no conozca previamente el código pueda entender qué hace cada parte del proyecto y cómo todas ellas se relacionan entre sí**.

Arquitectura general y puesta en marcha del sistema

Estructura general

El sistema **Kinema Nexus** se encuentra organizado en diferentes módulos, cada uno de ellos encargado de una etapa concreta del proceso. Aunque los distintos archivos trabajan conjuntamente, **la ejecución del sistema sigue una secuencia determinada**, comenzando por la calibración del operador y continuando posteriormente con la detección de movimiento y la generación de los datos destinados al robot.

Para que el sistema funcione correctamente, los archivos del proyecto deben mantenerse dentro de la estructura de carpetas establecida. Además, debe existir la carpeta destinada a las imágenes de referencia utilizadas durante la calibración, ya que estos recursos forman parte del proceso inicial de puesta en marcha.

La ejecución comienza mediante:

calibration.py

Este archivo constituye el **punto de entrada inicial del sistema**.

Primera etapa: calibración

Al ejecutar calibration.py, el sistema inicia el proceso de calibración del operador. Durante esta etapa se toman las medidas necesarias del usuario, obteniendo las referencias físicas que posteriormente serán utilizadas para adaptar el movimiento humano a las dimensiones del robot.

Para realizar esta calibración se utilizan las imágenes de referencia correspondientes, que deben encontrarse en la carpeta destinada a ellas dentro de la estructura del proyecto.

Una vez finalizado el proceso, los datos obtenidos se almacenan en un archivo:

calibration.json

Este archivo actúa como **resultado de la primera etapa y entrada para las siguientes**, evitando que las medidas tengan que volver a introducirse cada vez que se ejecuta el sistema.

El flujo inicial puede representarse como:

calibration.py

|



Medidas del operador

|



calibration.json

Segunda etapa: preparación del mapeo

Una vez generada la información de calibración, se utilizan los módulos encargados de preparar la correspondencia entre el operador y el robot.

En esta etapa interviene principalmente `robot_mapping.py`, que carga los datos almacenados en `calibration.json` y los relaciona con las dimensiones definidas para el robot. A partir de estas referencias se calculan los factores necesarios para establecer la relación de escala entre las dimensiones humanas y las dimensiones del robot.

De esta forma, la información obtenida durante la calibración pasa a convertirse en una configuración que podrá utilizarse durante el procesamiento del movimiento.

El proceso puede resumirse como:

`calibration.json`



`robot_mapping.py`



Factores de mapeo

Esta etapa se realiza antes de iniciar el procesamiento continuo de la cámara, de forma que el sistema ya dispone de las referencias necesarias para trabajar con el operador calibrado.

Tercera etapa: inicio de la detección de movimiento

Una vez realizada la calibración y preparada la información de mapeo, se inicia `human_pose_detection.py`.

Este archivo constituye el **punto de entrada del funcionamiento en tiempo real**. A partir de este momento, el sistema comienza a capturar imágenes continuamente mediante la cámara y a procesarlas para detectar la postura del operador.

La cámara proporciona cada frame a OpenCV, que posteriormente lo entrega al modelo YOLO Pose para realizar la detección de los puntos corporales.

Cámara



OpenCV



YOLO Pose





Puntos corporales detectados

Organización de los puntos corporales

Los puntos detectados por YOLO se incorporan posteriormente a la estructura propia del proyecto mediante Landmark.py y Pose.py.

Cada punto corporal se representa como un Landmark, almacenando sus coordenadas y su nivel de confianza. Estos puntos se agrupan posteriormente dentro de HumanPose, formando una representación estructurada de la pose completa del operador.

De esta manera, el sistema pasa de trabajar con los datos directamente proporcionados por el modelo de visión a trabajar con una estructura propia:

YOLO

|



Keypoints

|



Landmark

|



HumanPose

Esto permite que el resto de módulos trabajen independientemente del formato concreto utilizado por el modelo de detección.

Procesamiento de la pose

Una vez construida la pose humana, entra en funcionamiento la parte matemática del sistema.

relative_pose.py utiliza los puntos correspondientes al hombro, codo y muñeca para construir los vectores que representan el brazo y el antebrazo. Para realizar estos cálculos utiliza las funciones matemáticas proporcionadas por pose_math.py.

De esta forma, las coordenadas de los puntos detectados se transforman en información geométrica sobre la postura:

Hombro ———▶ Codo ———▶ Muñeca

|

|

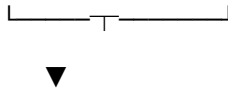


Brazo

Antebrazo

|

|



Vectores y ángulos

Esta transformación es importante porque el robot no necesita conocer únicamente dónde se encuentra un píxel dentro de la imagen, sino información relacionada con la orientación y configuración de los segmentos del brazo.

Restricciones del movimiento

Durante el procesamiento también se dispone del módulo `limits.py`, encargado de definir los límites de movimiento establecidos para las diferentes partes del sistema.

Estos valores permiten establecer las restricciones que posteriormente pueden aplicarse a los movimientos calculados. En el estado actual del proyecto, algunos de estos valores son todavía provisionales y deberán ajustarse cuando se defina completamente el robot y su configuración física.

Conversión a movimiento del robot

La información obtenida de la pose relativa llega finalmente a `robot_motion.py`.

Este módulo interpreta los vectores y ángulos calculados y realiza una primera conversión de la postura humana a valores correspondientes a las articulaciones del robot.

En la implementación actual esta conversión es todavía provisional y se centra principalmente en el brazo derecho. A partir de la orientación del brazo y del ángulo del codo se calculan los valores de las primeras articulaciones, mientras que las restantes permanecen actualmente a cero.

Posteriormente se aplican los límites establecidos mediante una función de restricción para evitar valores fuera del rango permitido.

Pose relativa



Robot Motion



J1 – J2 – J3 – J4 – J5 – J6

Generación del archivo de movimiento

Una vez calculada la configuración del robot, `robot_motion.py` genera el archivo:

`robot_motion.json`

Este archivo contiene los valores correspondientes a las articulaciones calculadas.

Por tanto, en el estado actual del proyecto, la salida de la cadena de procesamiento no consiste todavía en enviar directamente una orden física al robot, sino en **generar una representación estructurada del movimiento mediante JSON**.

Flujo completo de ejecución

Teniendo en cuenta todas las etapas anteriores, el funcionamiento general del sistema puede resumirse en dos grandes fases.

Fase 1: preparación del sistema

calibration.py

|



Medidas del operador

|



calibration.json

|



robot_mapping.py

|



Referencias y factores de mapeo

Esta primera fase prepara toda la información necesaria antes de comenzar la detección continua.

Fase 2: detección y generación de movimiento

human_pose_detection.py

|



Cámara

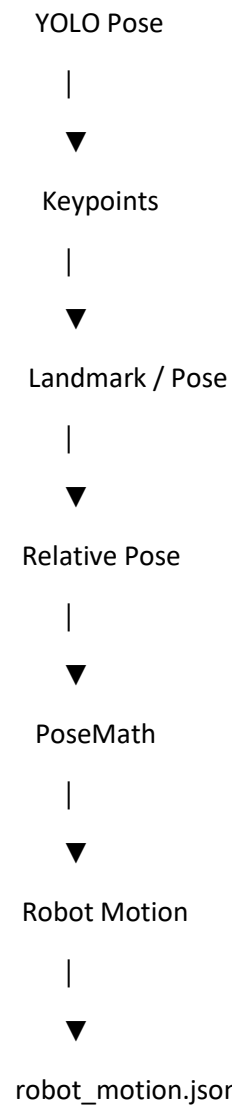
|



OpenCV

|





De esta forma, el sistema completo puede entenderse como una cadena en la que cada módulo recibe la información generada por el anterior y la transforma en un formato más útil para la siguiente etapa.

Papel de cada módulo dentro del sistema

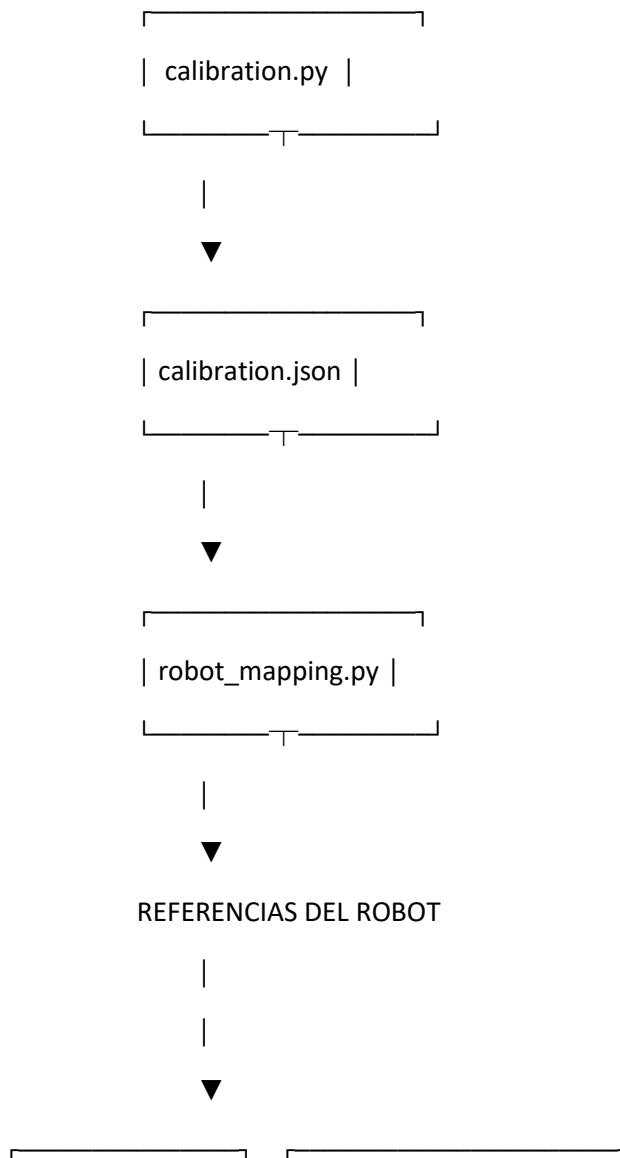
La función de cada archivo puede resumirse de la siguiente manera:

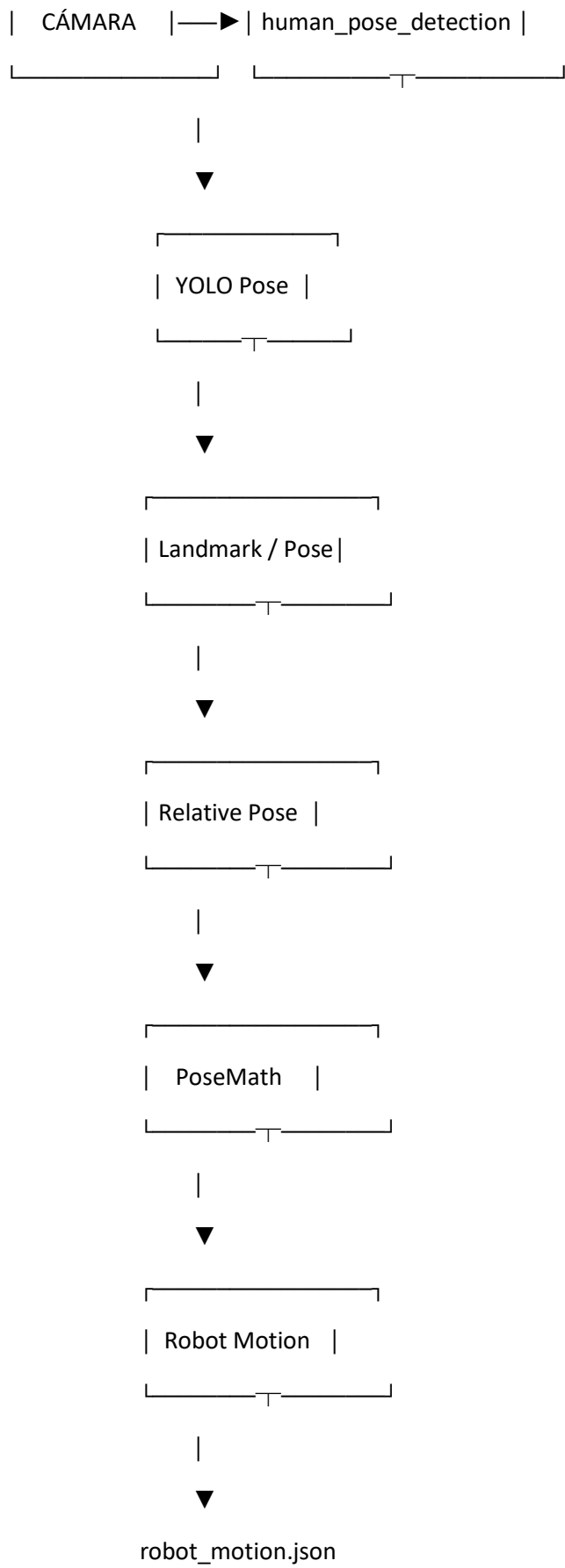
Módulo	Función dentro del sistema
calibration.py	Obtiene las medidas de referencia del operador y genera calibration.json.
pose_raw.py	Organiza la información de pose obtenida inicialmente.
landmark.py	Representa individualmente cada punto corporal detectado.
pose.py	Agrupar los landmarks para representar la pose humana.
pose_math.py	Realiza las operaciones matemáticas sobre los puntos y vectores.

Módulo	Función dentro del sistema
relative_pose.py	Calcula los vectores relativos de los brazos.
limits.py	Define las restricciones de movimiento.
robot_mapping.py	Relaciona las dimensiones del operador con las del robot.
robot_motion.py	Convierte la información de la pose en valores de articulaciones y genera robot_motion.json.
human_pose_detection.py	Coordina la ejecución en tiempo real de todo el proceso.
pose_drawer.py	Permite visualizar sobre la imagen la pose detectada.

Arquitectura conceptual

En conjunto, la arquitectura puede representarse mediante el siguiente esquema:





Estado actual de la integración

La arquitectura implementada permite recorrer actualmente el flujo completo desde la **calibración del operador y la detección de su movimiento hasta la generación de una configuración de articulaciones del robot**.

No obstante, determinadas partes continúan siendo provisionales. Las dimensiones del robot y algunos límites todavía deben ajustarse a la configuración física definitiva, el mapeo calculado todavía no se utiliza directamente dentro de la conversión actual de robot_motion.py, el movimiento se calcula actualmente a partir del brazo derecho y las articulaciones restantes permanecen a cero.

Por tanto, la implementación actual debe entenderse como una **primera integración funcional de la cadena percepción → procesamiento → generación de movimiento**, que establece la estructura necesaria para continuar posteriormente con la integración definitiva del robot.

Landmark.py

```
class Landmark:
```

```
    def __init__(self, x=0, y=0, confidence=0):
```

```
        self.x = x
```

```
        self.y = y
```

```
        self.confidence = confidence
```

```
    def __repr__(self):
```

```
        return f"({self.x:.1f}, {self.y:.1f}) conf={self.confidence:.2f}"
```


Landmark.py Explicación

Landmark

Landmark es una clase que utilizamos para **representar un punto corporal detectado por el sistema**.

Cuando el sistema de visión artificial detecta, por ejemplo, el hombro, el codo o la muñeca de una persona, necesita guardar la información obtenida. Para ello creamos un objeto Landmark.

Cada Landmark almacena tres datos:

- **x** → posición horizontal del punto en la imagen.
- **y** → posición vertical del punto en la imagen.
- **confidence** → nivel de confianza de la detección.

Por ejemplo, podríamos tener un punto como:

x = 320

y = 240

confidence = 0.95

Esto significaría que el punto se encuentra en la posición (320, 240) de la imagen y que el modelo tiene un **95 % de confianza** en esa detección.

__init__

```
def __init__(self, x=0, y=0, confidence=0):
```

```
    self.x = x
```

```
    self.y = y
```

```
    self.confidence = confidence
```

__init__ es el **constructor de la clase**.

Su función es definir qué información tendrá un Landmark cuando se cree.

Recibe tres parámetros:

x

x=0

Representa la **coordenada horizontal** del punto dentro de la imagen.

En una imagen, el eje X aumenta de izquierda a derecha:

(0,0) —————→ X

|

|

|

↓

Y

Por tanto, cuanto mayor sea x, más a la derecha estará el punto.

El valor 0 es el valor predeterminado que se utiliza si no proporcionamos una coordenada.

y

y=0

Representa la **coordenada vertical**.

En las imágenes digitales, normalmente el eje Y aumenta desde arriba hacia abajo.

Por tanto:

- y pequeño → punto cerca de la parte superior.
- y grande → punto más abajo.

confidence

confidence=0

Representa la **confianza que tiene el modelo en la detección**.

Normalmente trabajamos con valores entre 0 y 1.

Por ejemplo:

0.20 → detección poco fiable

0.50 → confianza media

0.95 → detección muy fiable

El valor predeterminado es 0, que utilizamos cuando todavía no tenemos una confianza concreta.

self.x, self.y y self.confidence

Dentro del constructor encontramos:

self.x = x

self.y = y

self.confidence = confidence

Aquí estamos **guardando los valores recibidos dentro del propio objeto**.

La diferencia entre x y self.x es importante para entender Python:

- x → es el parámetro que recibe la función.
- self.x → es el dato que pertenece al Landmark que acabamos de crear.

Por ejemplo:

```
landmark = Landmark(320, 240, 0.95)
```

crearía un objeto cuyo contenido sería conceptualmente:

Landmark

└─ x = 320

└─ y = 240

└─ confidence = 0.95

A partir de ese momento podemos acceder a esos datos mediante:

```
landmark.x
```

```
landmark.y
```

```
landmark.confidence
```

Esto será muy importante posteriormente, porque otras partes del proyecto podrán recibir un Landmark y utilizar directamente sus coordenadas.

`__repr__`

```
def __repr__(self):
```

```
    return f"({self.x:.1f}, {self.y:.1f}) conf={self.confidence:.2f}"
```

`__repr__` es un método especial de Python que determina **cómo queremos que se represente el objeto cuando lo mostramos**.

Sin este método, si intentáramos imprimir un Landmark, Python podría mostrar algo poco útil, relacionado con la dirección del objeto en memoria.

Nosotros queremos que sea fácilmente legible.

Por eso hemos definido:

```
(x, y) conf=confidence
```

Por ejemplo, si tenemos:

```
Landmark(320.456, 240.789, 0.9567)
```

la representación será:

```
(320.5, 240.8) conf=0.96
```

¿Qué significa `:.1f`?

En:

```
self.x:.1f
```

estamos indicando cómo queremos mostrar el número.

- `.1` → queremos **un decimal**.
- `f` → estamos trabajando con un número decimal (float).

Por eso:

320.456 → 320.5

Lo mismo ocurre con y.

¿Qué significa :.2f?

En:

self.confidence:.2f

indicamos que queremos mostrar **dos decimales**.

Por ejemplo:

0.9567 → 0.96

Esto hace que la información sea mucho más fácil de leer cuando estamos depurando el programa.

¿Por qué necesitamos Landmark?

Esta clase es una de las piezas básicas del sistema.

YOLO puede detectar puntos corporales, pero nosotros necesitamos **una estructura propia para trabajar con esos puntos**.

En lugar de tener continuamente tres variables separadas:

x

y

confidence

podemos tener:

landmark

y dentro de él:

landmark.x

landmark.y

landmark.confidence

Esto nos permite construir posteriormente estructuras más complejas.

Por ejemplo, podremos tener una Pose formada por diferentes Landmark:

Pose

|

|— Shoulder → Landmark

|— Elbow → Landmark

|— Wrist → Landmark

Y a partir de ahí, otras partes del proyecto podrán utilizar esos puntos para realizar cálculos matemáticos, como **vectores, distancias y ángulos**.

Por tanto, Landmark no realiza ningún cálculo de visión artificial. Su función es mucho más sencilla: **crear una representación ordenada de un punto corporal detectado y guardar su posición y confianza.**

Pose.py

```
from landmark import Landmark
```

```
class HumanPose:
```

```
    def __init__(self):
```

```
        self.nose = Landmark()
```

```
        self.left_eye = Landmark()
```

```
        self.right_eye = Landmark()
```

```
        self.left_ear = Landmark()
```

```
        self.right_ear = Landmark()
```

```
        self.left_shoulder = Landmark()
```

```
        self.right_shoulder = Landmark()
```

```
        self.left_elbow = Landmark()
```

```
        self.right_elbow = Landmark()
```

```
        self.left_wrist = Landmark()
```

```
        self.right_wrist = Landmark()
```

```
        self.left_hip = Landmark()
```

```
        self.right_hip = Landmark()
```

```
self.left_knee = Landmark()
self.right_knee = Landmark()

self.left_ankle = Landmark()
self.right_ankle = Landmark()
def print_pose(self):

    print("LEFT SHOULDER :", self.left_shoulder)
    print("RIGHT SHOULDER:", self.right_shoulder)

    print("LEFT ELBOW :", self.left_elbow)
    print("RIGHT ELBOW:", self.right_elbow)

    print("LEFT WRIST :", self.left_wrist)
    print("RIGHT WRIST:", self.right_wrist)

def print_all(self):

    for nombre, landmark in self.__dict__.items():
        print(f"{nombre:15} -> {landmark}")
def get_arm_landmarks(self):

#Devuelve únicamente las articulaciones de ambos brazos.

    return {
        "left_shoulder": self.left_shoulder,
        "right_shoulder": self.right_shoulder,
        "left_elbow": self.left_elbow,
        "right_elbow": self.right_elbow,
```

```
"left_wrist": self.left_wrist,  
"right_wrist": self.right_wrist,  
}
```

Pose.py explicación

El archivo pose.py define la clase HumanPose, que se encarga de **representar la postura completa de una persona mediante los puntos corporales (Landmark) detectados por el sistema**.

Mientras que Landmark representa **un único punto** del cuerpo, HumanPose agrupa todos esos puntos dentro de una misma estructura.

De esta forma, una postura humana queda organizada de la siguiente manera:

HumanPose

```
|  
|  
| └─ Cabeza  
|  
|   └─ Nose  
|  
|   └─ Left Eye  
|  
|   └─ Right Eye  
|  
|   └─ Left Ear  
|  
|   └─ Right Ear  
|  
|  
| └─ Brazos  
|  
|   └─ Left Shoulder  
|  
|   └─ Right Shoulder  
|  
|   └─ Left Elbow  
|  
|   └─ Right Elbow  
|  
|   └─ Left Wrist  
|  
|   └─ Right Wrist  
|  
|  
| └─ Cadera  
|  
|   └─ Left Hip  
|  
|   └─ Right Hip  
|
```

└─ Piernas

├─ Left Knee

├─ Right Knee

├─ Left Ankle

└─ Right Ankle

Cada uno de estos puntos es un objeto de la clase Landmark que hemos explicado anteriormente.

Por tanto, HumanPose funciona como un **contenedor organizado de todos los puntos corporales que nos interesan**.

Importación de Landmark

```
from landmark import Landmark
```

Antes de poder utilizar la clase Landmark, necesitamos importarla.

Landmark se encuentra en otro archivo del proyecto, landmark.py.

Esta línea permite que pose.py pueda crear y utilizar objetos Landmark.

La relación entre ambos archivos es, por tanto:

landmark.py

|

| define

↓

Landmark

|

| utilizado por

↓

pose.py

|

↓

HumanPose

Esto es importante porque HumanPose **no crea una nueva forma de representar los puntos corporales**. Utiliza la estructura Landmark que ya hemos definido anteriormente.

Clase HumanPose

```
class HumanPose:
```

Aquí definimos la clase HumanPose.

El nombre puede traducirse como “**postura humana**”.

Su función es crear un objeto capaz de almacenar todos los puntos corporales de una persona.

Cada vez que creemos un objeto HumanPose, tendremos disponible una estructura completa para guardar la información de la persona detectada.

`__init__`

```
def __init__(self):
```

Este es el constructor de HumanPose.

A diferencia de Landmark, aquí no necesitamos recibir las coordenadas como parámetros.

Esto se debe a que inicialmente simplemente queremos **crear la estructura que contendrá todos los puntos**.

Los valores de cada punto comienzan utilizando:

```
Landmark()
```

Como vimos anteriormente, si no proporcionamos ningún valor, Landmark utiliza:

```
x = 0
```

```
y = 0
```

```
confidence = 0
```

Por tanto, al crear una nueva HumanPose, todos sus puntos existen, pero inicialmente están vacíos o sin información de detección.

Puntos de la cabeza

```
self.nose = Landmark()
```

```
self.left_eye = Landmark()
```

```
self.right_eye = Landmark()
```

```
self.left_ear = Landmark()
```

```
self.right_ear = Landmark()
```

Aquí creamos los Landmark correspondientes a la cabeza:

- nose → nariz.
- left_eye → ojo izquierdo.
- right_eye → ojo derecho.
- left_ear → oreja izquierda.

- `right_ear` → oreja derecha.

Cada uno de ellos es independiente.

Por ejemplo:

```
self.left_eye
```

contiene su propio `Landmark`, con sus propias coordenadas y su propia confianza.

Puntos de los hombros

```
self.left_shoulder = Landmark()
```

```
self.right_shoulder = Landmark()
```

Representan los dos hombros:

- `left_shoulder` → hombro izquierdo.
- `right_shoulder` → hombro derecho.

Estos puntos serán especialmente importantes posteriormente porque forman parte de la información necesaria para analizar el movimiento de los brazos.

Puntos de los codos

```
self.left_elbow = Landmark()
```

```
self.right_elbow = Landmark()
```

Representan los dos codos.

Al igual que ocurre con los hombros, estos puntos tendrán posteriormente un papel importante en los cálculos matemáticos del movimiento.

Por ejemplo, podremos utilizar:

Hombro → Codo

para construir un vector que represente el primer segmento del brazo.

Puntos de las muñecas

```
self.left_wrist = Landmark()
```

```
self.right_wrist = Landmark()
```

Representan las dos muñecas.

Los tres puntos:

Shoulder



Elbow



Wrist

permitirán posteriormente representar matemáticamente cada brazo mediante diferentes vectores y calcular, entre otras cosas, los ángulos de las articulaciones.

Puntos de la cadera

```
self.left_hip = Landmark()
```

```
self.right_hip = Landmark()
```

Representan la cadera izquierda y derecha.

Aunque actualmente el análisis principal del proyecto se centra en los brazos, estos puntos forman parte de la estructura general de la postura humana.

Puntos de las piernas

```
self.left_knee = Landmark()
```

```
self.right_knee = Landmark()
```

```
self.left_ankle = Landmark()
```

```
self.right_ankle = Landmark()
```

Finalmente se crean los puntos correspondientes a:

- rodilla izquierda.
- rodilla derecha.
- tobillo izquierdo.
- tobillo derecho.

De esta manera, HumanPose contiene una representación bastante completa de la postura corporal.

print_pose()

```
def print_pose(self):
```

```
    print("LEFT SHOULDER :", self.left_shoulder)
```

```
    print("RIGHT SHOULDER:", self.right_shoulder)
```

```
print("LEFT ELBOW :", self.left_elbow)
print("RIGHT ELBOW:", self.right_elbow)
```

```
print("LEFT WRIST :", self.left_wrist)
print("RIGHT WRIST:", self.right_wrist)
```

Esta función sirve para **mostrar por pantalla la información de los principales puntos de los brazos**.

No modifica ningún dato de la postura.

Simplemente obtiene los Landmark correspondientes a:

Hombros

Codos

Muñecas

y los muestra utilizando la representación que hemos definido anteriormente en `Landmark.__repr__()`.

Por ejemplo, podríamos obtener:

LEFT SHOULDER : (320.5, 240.8) conf=0.96

RIGHT SHOULDER: (450.2, 242.1) conf=0.94

LEFT ELBOW : (300.1, 350.7) conf=0.91

RIGHT ELBOW: (470.4, 351.2) conf=0.93

LEFT WRIST : (280.3, 450.5) conf=0.89

RIGHT WRIST: (500.7, 449.8) conf=0.90

Esta función resulta especialmente útil durante el desarrollo para **comprobar rápidamente qué está detectando el sistema en los brazos**.

`print_all()`

```
def print_all(self):
```

```
    for nombre, landmark in self.__dict__.items():
        print(f"{nombre:15} -> {landmark}")
```

Esta función tiene un objetivo parecido a `print_pose()`, pero en este caso **muestra todos los puntos almacenados dentro de HumanPose**.

La diferencia fundamental es que `print_pose()` selecciona manualmente los puntos de los brazos, mientras que `print_all()` recorre automáticamente todos los atributos de la clase.

self.__dict__

self.__dict__ es una estructura interna de Python que contiene los atributos que pertenecen al objeto.

En nuestro caso, contiene elementos como:

nose

left_eye

right_eye

left_shoulder

right_shoulder

left_elbow

...

La función recorre estos elementos y muestra el nombre del punto junto con su Landmark.

El resultado sería aproximadamente:

nose -> (320.0, 120.0) conf=0.98

left_eye -> (310.0, 100.0) conf=0.97

right_eye -> (330.0, 100.0) conf=0.97

left_shoulder -> (290.0, 240.0) conf=0.95

right_shoulder -> (450.0, 240.0) conf=0.94

...

La ventaja de hacerlo de esta manera es que **no necesitamos escribir manualmente cada punto que queremos mostrar**.

get_arm_landmarks()

def get_arm_landmarks(self):

Esta función es especialmente importante porque empieza a mostrar cómo HumanPose se conecta con el resto del sistema.

Su objetivo es **extraer únicamente los puntos que pertenecen a los brazos**.

La postura completa contiene muchos puntos:

Cabeza

Brazos

Cadera

Piernas

pero para determinadas operaciones no necesitamos toda esa información.

En nuestro proyecto nos interesa trabajar específicamente con:

Left Shoulder

Right Shoulder

Left Elbow

Right Elbow

Left Wrist

Right Wrist

Por eso la función devuelve únicamente esos seis Landmark.

El return

```
return {  
    "left_shoulder": self.left_shoulder,  
    "right_shoulder": self.right_shoulder,  
    "left_elbow": self.left_elbow,  
    "right_elbow": self.right_elbow,  
    "left_wrist": self.left_wrist,  
    "right_wrist": self.right_wrist,  
}
```

Aquí creamos y devolvemos un **diccionario de Python**.

Un diccionario permite almacenar información utilizando una relación:

clave → valor

En nuestro caso:

"left_shoulder" → Landmark del hombro izquierdo

"right_shoulder" → Landmark del hombro derecho

"left_elbow" → Landmark del codo izquierdo

...

Por tanto, cuando otra parte del programa llame a:

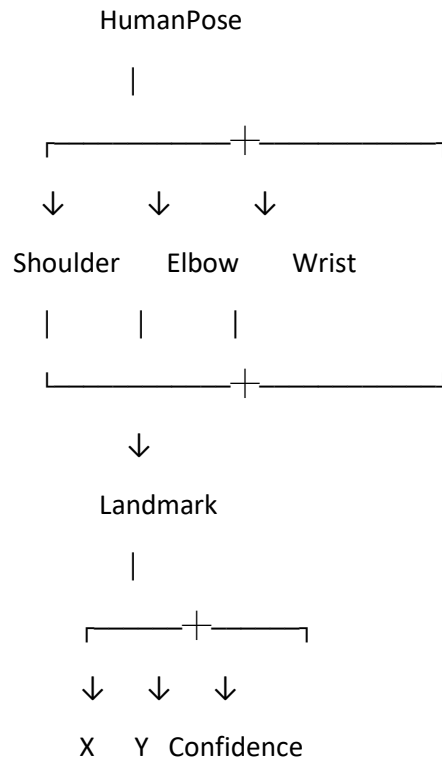
```
pose.get_arm_landmarks()
```

recibirá únicamente la información relacionada con los brazos.

Esto permite que las siguientes partes del sistema no tengan que conocer o recorrer toda la estructura de HumanPose cuando solamente necesitan trabajar con los brazos.

Relación entre Landmark y HumanPose

En este punto ya podemos ver claramente cómo se construyen las primeras piezas del proyecto:



Landmark es la **unidad básica**: representa un único punto.

HumanPose es la **estructura que agrupa todos esos puntos** para representar una postura humana completa.

Más adelante, otras partes del proyecto podrán coger los Landmark de hombros, codos y muñecas y utilizarlos para realizar los cálculos necesarios para el control del movimiento.

Por tanto, podemos resumir el papel de este archivo como:

pose.py organiza los puntos corporales detectados y los agrupa en una estructura que representa la postura humana. Además, proporciona funciones para consultar y mostrar esta información y para extraer específicamente los puntos de los brazos que utilizarán las siguientes etapas del sistema.

pose_math.py

```
import math
```

```
# =====
```

```
# VECTORES
```

```
# =====
```

```
def calculate_vector(origin, destination):
```

```
    dx = destination.x - origin.x
```

```
    dy = destination.y - origin.y
```

```
    return dx, dy
```

```
def normalize_vector(vector):
```

```
    modulo = math.sqrt(
```

```
        vector[0]**2 +
```

```
        vector[1]**2
```

```
    )
```

```
    if modulo == 0:
```

```
        return (0, 0)
```

```
    vector_normalizado = (
```

```
        vector[0] / modulo,
```

```
        vector[1] / modulo
```

```
    )
```

```
    return vector_normalizado
```



```
# =====  
# ÁNGULOS  
# =====
```

```
def calculate_angle(vector_A, vector_B):
```

```
    # Producto escalar
```

```
    producto_escalar = (  
        vector_A[0] * vector_B[0] +  
        vector_A[1] * vector_B[1]  
    )
```

```
    # Módulos
```

```
    modulo_vector_A = math.sqrt(  
        vector_A[0]**2 +  
        vector_A[1]**2  
    )
```

```
    modulo_vector_B = math.sqrt(  
        vector_B[0]**2 +  
        vector_B[1]**2  
    )
```

```
    if modulo_vector_A == 0 or modulo_vector_B == 0:  
        return 0
```

```
    # Coseno del ángulo
```

```
    coseno_angulo = (  
        producto_escalar /  
        (modulo_vector_A * modulo_vector_B)  
    )
```

```

# Evita errores numéricos
coseno_angulo = max(-1.0, min(1.0, coseno_angulo))

# Ángulo en grados
angulo = math.degrees(math.acos(coseno_angulo))

return angulo

# =====
# DISTANCIAS
# =====

def calculate_distance(origin, destination):

    dx = destination.x - origin.x
    dy = destination.y - origin.y

    distancia = math.sqrt(
        dx**2 +
        dy**2
    )

    return distancia

```

pose_math.py explicación

El archivo pose_math.py contiene las **operaciones matemáticas utilizadas para analizar los puntos corporales detectados**.

En los archivos anteriores hemos definido qué es un Landmark y cómo podemos agrupar varios Landmark dentro de una HumanPose. En este archivo damos el siguiente paso: utilizamos las coordenadas de esos puntos para obtener información matemática sobre la postura.

Actualmente, el archivo realiza tres tipos principales de operaciones:

1. **Vectores** → permiten representar la dirección entre dos puntos.

2. **Ángulos** → permiten conocer el ángulo formado entre dos segmentos.
3. **Distancias** → permiten conocer la separación entre dos puntos.

Estas operaciones serán fundamentales posteriormente para analizar los brazos de la persona.

Importación de math

```
import math
```

math es una librería incluida en Python que proporciona diferentes funciones matemáticas.

En este archivo utilizamos principalmente:

- `math.sqrt()` → calcula una raíz cuadrada.
- `math.acos()` → calcula el arco coseno.
- `math.degrees()` → convierte un ángulo de radianes a grados.

No necesitamos implementar nosotros estas operaciones matemáticas porque Python ya proporciona las herramientas necesarias.

Vectores

`calculate_vector()`

```
def calculate_vector(origin, destination):
```

```
    dx = destination.x - origin.x
```

```
    dy = destination.y - origin.y
```

```
    return dx, dy
```

Esta función calcula el **vector que va desde un punto de origen hasta un punto de destino**.

Recibe dos Landmark:

- `origin` → punto desde el que comienza el vector.
- `destination` → punto al que llega el vector.

Como los Landmark contienen las coordenadas x e y, podemos utilizar directamente:

```
origin.x
```

```
origin.y
```

```
destination.x
```

```
destination.y
```

¿Qué es un vector?

Un vector puede utilizarse para representar un desplazamiento entre dos puntos.

Por ejemplo, si tenemos:

Origen $\rightarrow (100, 200)$

Destino $\rightarrow (150, 300)$

el vector se obtiene restando las coordenadas:

$$\vec{v} = (x_{\text{destino}} - x_{\text{origen}}, y_{\text{destino}} - y_{\text{origen}}) \quad \vec{v} = (x_{\text{destino}} - x_{\text{origen}}, y_{\text{destino}} - y_{\text{origen}})$$

Por tanto:

$$\vec{v} = (150 - 100, 300 - 200) \quad \vec{v} = (150 - 100, 300 - 200) \quad \vec{v} = (50, 100) \quad \vec{v} = (50, 100)$$

Esto significa que para llegar desde el origen hasta el destino debemos desplazarnos:

- 50 píxeles horizontalmente.
- 100 píxeles verticalmente.

¿Cómo lo hace el código?

Primero calculamos la diferencia horizontal:

$$dx = \text{destination.x} - \text{origin.x}$$

Después calculamos la diferencia vertical:

$$dy = \text{destination.y} - \text{origin.y}$$

Finalmente:

return dx, dy

devuelve ambas componentes del vector.

El resultado será una pareja de valores:

(dx, dy)

Por ejemplo:

(50, 100)

En Kinema Nexus podemos utilizar esto para representar segmentos corporales.

Por ejemplo:

Hombro $\longrightarrow$ Codo

puede convertirse matemáticamente en un vector:

$$\vec{v}_{\text{hombro-codo}} \quad \vec{v}_{\text{hombro-codo}}$$

Y de la misma manera:

Codo —→ Muñeca

puede convertirse en otro vector:

$\vec{v}_{\text{codo-muñeca}}$

Esto será especialmente importante para calcular posteriormente el ángulo del codo.

normalize_vector()

def normalize_vector(vector):

```
    modulo = math.sqrt(
        vector[0]**2 +
        vector[1]**2
    )
```

```
    if modulo == 0:
        return (0, 0)
```

```
    vector_normalizado = (
        vector[0] / modulo,
        vector[1] / modulo
    )
```

```
    return vector_normalizado
```

Esta función recibe un vector y calcula su **vector normalizado**.

¿Qué significa normalizar un vector?

Normalizar un vector significa conservar **su dirección**, pero hacer que su longitud sea exactamente 1.

Por ejemplo, imaginemos:

$\vec{v}=(3,4)$

Su longitud es:

$|\vec{v}|=\sqrt{3^2+4^2}$ $|\vec{v}|=\sqrt{9+16}$ $|\vec{v}|=5$

Para normalizarlo dividimos cada componente entre su módulo:

$$\vec{v} = (35, 45) \hat{v} = \left(\frac{3}{5}, \frac{4}{5} \right)$$

Por tanto:

$$\vec{v} = (0.6, 0.8) \hat{v} = (0.6, 0.8)$$

La dirección sigue siendo la misma, pero ahora la longitud del vector es 1.

Cálculo del módulo

El código utiliza:

```
modulo = math.sqrt(
    vector[0]**2 +
    vector[1]**2
)
```

Aquí estamos aplicando el **teorema de Pitágoras**.

Si el vector es:

(x, y)

su longitud se calcula como:

$$|\vec{v}| = \sqrt{x^2 + y^2} \quad |\vec{v}| = \sqrt{x^2 + y^2}$$

Por eso:

vector[0]

representa la componente x, mientras que:

vector[1]

representa la componente y.

¿Por qué comprobamos modulo == 0?

if modulo == 0:

return (0, 0)

Un vector puede tener longitud cero cuando sus dos puntos coinciden.

Por ejemplo:

Origen (100,100)

Destino (100,100)

El vector sería:

(0,0)(0,0)

y su módulo sería 0.

No podemos dividir un vector entre 0, por lo que devolvemos (0,0) para evitar una división inválida.

Normalización

Finalmente:

```
vector_normalizado = (  
    vector[0] / modulo,  
    vector[1] / modulo  
)
```

divide cada componente entre el módulo.

El resultado es un nuevo vector que mantiene la dirección original, pero cuya longitud es 1.

Ángulos

calculate_angle()

```
def calculate_angle(vector_A, vector_B):
```

Esta función calcula el **ángulo existente entre dos vectores**.

Recibe:

- vector_A → primer vector.
- vector_B → segundo vector.

Esta función es especialmente importante para nuestro proyecto porque permite obtener el ángulo de una articulación.

Por ejemplo, para estudiar un codo podemos utilizar dos segmentos:

Hombro — Codo — Muñeca

↑

articulación

Representamos esos segmentos mediante vectores y calculamos el ángulo que forman.

Producto escalar

La primera operación que realiza la función es:

```
producto_escalar = (  
    vector_A[0] * vector_B[0] +
```

```
vector_A[1] * vector_B[1]
)
```

Esto calcula el **producto escalar** de los dos vectores.

Si tenemos:

$$\vec{A}=(A_x,A_y)\vec{A}=(A_x,A_y)$$

y

$$\vec{B}=(B_x,B_y)\vec{B}=(B_x,B_y)$$

el producto escalar se calcula como:

$$\vec{A}\cdot\vec{B}=A_xB_x+A_yB_y\vec{A}\cdot\vec{B}=A_xB_x+A_yB_y$$

Por ejemplo:

$$\vec{A}=(2,3)\vec{A}=(2,3) \vec{B}=(4,5)\vec{B}=(4,5)$$

entonces:

$$\vec{A}\cdot\vec{B}=2\cdot4+3\cdot5\vec{A}\cdot\vec{B}=2\cdot4+3\cdot5=8+15=23=23$$

El producto escalar es necesario porque nos permite obtener el ángulo entre ambos vectores.

Módulos de los vectores

A continuación calculamos la longitud de cada vector:

```
modulo_vector_A = math.sqrt(
    vector_A[0]**2 +
    vector_A[1]**2
)
```

y:

```
modulo_vector_B = math.sqrt(
    vector_B[0]**2 +
    vector_B[1]**2
)
```

Utilizamos nuevamente la fórmula:

$$|\vec{v}|=\sqrt{x^2+y^2}|\vec{v}|=\sqrt{x^2+y^2}$$

Por tanto, obtenemos:

$$|\vec{A}||\vec{A}|$$

y:

$$|\vec{B}| |\vec{B}|$$

Evitar vectores de longitud cero

if modulo_vector_A == 0 or modulo_vector_B == 0:

return 0

Para calcular el ángulo necesitamos dividir entre los módulos de los vectores.

Si alguno de ellos es 0, estaríamos intentando dividir entre cero.

Por eso, si uno de los vectores no tiene longitud, la función devuelve 0 y evita realizar el cálculo.

Cálculo del coseno

Después utilizamos:

```

coseno_angulo = (
    producto_escalar /
    (modulo_vector_A * modulo_vector_B)
)

```

Esta operación procede de la fórmula matemática del producto escalar:

$$\vec{A} \cdot \vec{B} = |\vec{A}| |\vec{B}| \cos(\theta) \quad \vec{A} \cdot \vec{B} = |\vec{A}| |\vec{B}| \cos(\theta)$$

Despejando el coseno:

$$\cos(\theta) = \frac{\vec{A} \cdot \vec{B}}{|\vec{A}| |\vec{B}|} \quad \cos(\theta) = \frac{\vec{A} \cdot \vec{B}}{|\vec{A}| |\vec{B}|}$$

Eso es exactamente lo que estamos implementando en el código.

En este punto ya conocemos el **coseno del ángulo**, pero todavía no tenemos el ángulo propiamente dicho.

Protección frente a errores numéricos

```

coseno_angulo = max(-1.0, min(1.0, coseno_angulo))

```

Matemáticamente, el coseno siempre debe estar entre:

$$-1 \leq \cos(\theta) \leq 1$$

Sin embargo, debido a pequeños errores de precisión que pueden aparecer al trabajar con números decimales, Python podría obtener ocasionalmente un valor ligeramente superior a 1 o inferior a -1.

Por ejemplo:

1.0000000001

Aunque matemáticamente debería ser 1.

Para evitar problemas posteriores al calcular el arco coseno, limitamos el resultado al intervalo válido:

$[-1, 1]$

Esta línea garantiza que el valor utilizado posteriormente siempre sea válido.

Obtención del ángulo

Finalmente:

```
angulo = math.degrees(math.acos(coseno_angulo))
```

Aquí realizamos dos operaciones.

math.acos()

acos es el **arco coseno**.

Hasta este momento tenemos:

$\cos(\theta)$

y queremos obtener:

θ

Por eso utilizamos:

```
math.acos(coseno_angulo)
```

El resultado de `acos()` se proporciona en **radianes**.

math.degrees()

Nosotros queremos trabajar con ángulos en grados porque son más intuitivos para interpretar una articulación.

Por eso utilizamos:

```
math.degrees(...)
```

para convertir el resultado de radianes a grados.

Finalmente:

```
return angulo
```

devuelve el ángulo calculado.

Por ejemplo:

45°

90°

120°

180°

Distancias

`calculate_distance()`

```
def calculate_distance(origin, destination):
```

```
    dx = destination.x - origin.x
```

```
    dy = destination.y - origin.y
```

```
    distancia = math.sqrt(
```

```
        dx**2 +
```

```
        dy**2
```

```
    )
```

```
    return distancia
```

Esta función calcula la **distancia entre dos Landmark**.

Recibe:

- origin → punto de origen.
- destination → punto de destino.

Primero calcula la diferencia entre sus coordenadas:

```
dx = destination.x - origin.x
```

```
dy = destination.y - origin.y
```

Después utiliza nuevamente el teorema de Pitágoras:

$$d = \sqrt{dx^2 + dy^2}$$

Por ejemplo, si tenemos:

Origen = (100, 100)

Destino = (130, 140)

entonces:

$dx=30$ $dy=40$

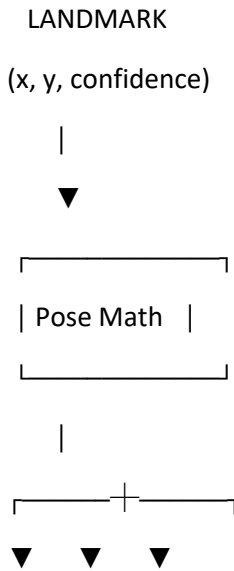
y:

$$d = \sqrt{30^2 + 40^2} = \sqrt{900 + 1600} = \sqrt{2500} = 50$$

La distancia entre ambos puntos es, por tanto, **50 píxeles**.

Resumen de pose_math.py

Este archivo transforma las coordenadas de los Landmark en información matemática que puede utilizar el resto del sistema.



Vectores Ángulos Distancias

Sus funciones principales son:

Función	Qué obtiene
calculate_vector()	Vector entre dos puntos
normalize_vector()	Vector con longitud 1 manteniendo su dirección
calculate_angle()	Ángulo entre dos vectores
calculate_distance()	Distancia entre dos puntos

Por tanto, pose_math.py constituye la **capa matemática básica del análisis de la postura**.

A partir de los Landmark proporcionados por HumanPose, permite convertir simples coordenadas de imagen en información que posteriormente puede utilizarse para interpretar el movimiento de los brazos.

Pose_drawer.py

```
import cv2

from pose_math import calculate_vector, calculate_angle#, calculate_distance

# =====
# LANDMARKS
# =====

def draw_landmark(img, landmark, color):

    if landmark.confidence > 0.3:

        cv2.circle(
            img,
            (int(landmark.x), int(landmark.y)),
            6,
            color,
            -1
        )

# =====
# CONNECTIONS
# =====

def draw_connection(img, p1, p2, color=(255, 255, 255)):

    if p1.confidence > 0.3 and p2.confidence > 0.3:

        cv2.line(
            img,
```

```
(int(p1.x), int(p1.y)),  
(int(p2.x), int(p2.y)),  
color,  
2  
)
```

```
# =====
```

```
# ANGLES
```

```
# =====
```

```
def draw_angles(img, pose):
```

```
# -----
```

```
# Brazo izquierdo
```

```
# -----
```

```
brazo_izquierdo = calculate_vector(  
    pose.left_elbow,  
    pose.left_shoulder  
)
```

```
antebrazo_izquierdo = calculate_vector(  
    pose.left_elbow,  
    pose.left_wrist  
)
```

```
angulo_izquierdo = calculate_angle(  
    brazo_izquierdo,  
    antebrazo_izquierdo  
)
```

```
cv2.putText(
    img,
    f"{angulo_izquierdo:.1f}°",
    (
        int(pose.left_elbow.x + 10),
        int(pose.left_elbow.y - 10)
    ),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.6,
    (0, 255, 255),
    2
)
```

```
# -----
```

```
# Brazo derecho
```

```
# -----
```

```
brazo_derecho = calculate_vector(
    pose.right_elbow,
    pose.right_shoulder
)
```

```
antebrazo_derecho = calculate_vector(
    pose.right_elbow,
    pose.right_wrist
)
```

```
angulo_derecho = calculate_angle(
    brazo_derecho,
    antebrazo_derecho
)
```

```

cv2.putText(
    img,
    f"{angulo_derecho:.1f}°",
    (
        int(pose.right_elbow.x + 10),
        int(pose.right_elbow.y - 10)
    ),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.6,
    (0, 255, 255),
    2
)

```

```

# =====
# DISTANCES
# =====

```

```

#def draw_distances(img, pose):

```

```

# -----
# Brazo izquierdo
# -----

```

```

#distancia_brazo_izquierdo = calculate_distance(
#   pose.left_shoulder,
#   pose.left_elbow
#)

```

```

#distancia_antebrazo_izquierdo = calculate_distance(
#   pose.left_elbow,

```



```

# pose.left_wrist
#)

# -----
# Brazo derecho
# -----

#distancia_brazo_derecho = calculate_distance(
# pose.right_shoulder,
# pose.right_elbow
#)

#distancia_antebrazo_derecho = calculate_distance(
# pose.right_elbow,
# pose.right_wrist
#)

# -----
# Mostrar distancias
# -----

#cv2.putText(img,f"Brazo Izquierdo: {distancia_brazo_izquierdo:.1f}px",(20,
30),cv2.FONT_HERSHEY_SIMPLEX,0.6,(255, 255, 255),2)

#cv2.putText(img,f"Antebrazp Izquierdo: {distancia_antebrazo_izquierdo:.1f}px",(20,
55),cv2.FONT_HERSHEY_SIMPLEX,0.6,(255, 255, 255),2)

#cv2.putText(img,f"Brazo Derecho: {distancia_brazo_derecho:.1f}px",(20,
80),cv2.FONT_HERSHEY_SIMPLEX,0.6,(255, 255, 255),2)

#cv2.putText(img,f"Antebrazo Derecho: {distancia_antebrazo_derecho:.1f}px",(20,
105),cv2.FONT_HERSHEY_SIMPLEX,0.6,(255, 255, 255),2)

```

```
# =====
```

```
# DRAW COMPLETE POSE
```

```
# =====
```

```
def draw_pose(frame, pose):
```

```
    frame_draw = frame.copy()
```

```
    # -----
```

```
    # Hombros
```

```
    # -----
```

```
    draw_landmark(
```

```
        frame_draw,
```

```
        pose.left_shoulder,
```

```
        (0, 0, 255)
```

```
    )
```

```
    draw_landmark(
```

```
        frame_draw,
```

```
        pose.right_shoulder,
```

```
        (0, 0, 255)
```

```
    )
```

```
    # -----
```

```
    # Codos
```

```
    # -----
```

```
    draw_landmark(
```

```
        frame_draw,
```

```
    pose.left_elbow,  
    (0, 255, 0)  
)
```

```
draw_landmark(  
    frame_draw,  
    pose.right_elbow,  
    (0, 255, 0)  
)
```

```
# -----  
# Muñecas  
# -----
```

```
draw_landmark(  
    frame_draw,  
    pose.left_wrist,  
    (255, 0, 0)  
)
```

```
draw_landmark(  
    frame_draw,  
    pose.right_wrist,  
    (255, 0, 0)  
)
```

```
# -----  
# Brazo izquierdo  
# -----
```

```
draw_connection(  

```

```
    frame_draw,  
    pose.left_shoulder,  
    pose.left_elbow  
)
```

```
draw_connection(  
    frame_draw,  
    pose.left_elbow,  
    pose.left_wrist  
)
```

```
# -----  
# Brazo derecho  
# -----
```

```
draw_connection(  
    frame_draw,  
    pose.right_shoulder,  
    pose.right_elbow  
)
```

```
draw_connection(  
    frame_draw,  
    pose.right_elbow,  
    pose.right_wrist  
)
```

```
# -----  
# Ángulos  
# -----
```

```
draw_angles(  
    frame_draw,  
    pose  
)
```

```
# -----  
# Distancias  
# -----
```

```
#draw_distances(  
#   frame_draw,  
#   pose  
#)
```

```
# -----  
# Mostrar resultado  
# -----
```

```
cv2.imshow(  
    "KINEMA NEXUS",  
    frame_draw  
)
```

Pose_drawer.py explicación

El archivo pose_drawer.py se encarga de representar visualmente sobre la imagen los puntos corporales detectados, las conexiones entre ellos y los ángulos de los brazos.

Hasta este momento tenemos:

- Landmark → representa cada punto corporal.
- HumanPose → organiza todos los puntos del cuerpo.
- pose_math.py → realiza las operaciones matemáticas necesarias para obtener vectores, ángulos y distancias.

pose_drawer.py utiliza toda esta información para mostrarla directamente sobre la imagen de la cámara.

De esta forma, podemos comprobar visualmente qué está detectando el sistema y qué cálculos está realizando.

Importación de librerías

El archivo comienza con:

```
import cv2
```

```
from pose_math import calculate_vector, calculate_angle
```

La primera línea importa OpenCV mediante cv2.

OpenCV es la librería que utilizamos para trabajar con imágenes y vídeo. En este archivo se utiliza principalmente para dibujar elementos sobre la imagen y mostrar el resultado.

Entre las funciones de OpenCV utilizadas encontramos:

- cv2.circle() → dibuja círculos.
- cv2.line() → dibuja líneas.
- cv2.putText() → escribe texto sobre una imagen.
- cv2.imshow() → muestra una imagen en una ventana.

La segunda línea importa dos funciones que ya hemos explicado en pose_math.py:

- calculate_vector() → calcula un vector entre dos puntos.
- calculate_angle() → calcula el ángulo entre dos vectores.

Por tanto, pose_drawer.py no vuelve a programar estas operaciones matemáticas, sino que **reutiliza las funciones que ya hemos creado anteriormente**.

La relación sería:

```
pose_math.py
```

↓

```
calculate_vector()
```

calculate_angle()

↓

pose_drawer.py

↓

Representación visual

draw_landmark()

La primera función es:

```
def draw_landmark(img, landmark, color):
```

Su función es **dibujar un punto corporal sobre la imagen**.

Recibe tres elementos:

- img → imagen sobre la que queremos dibujar.
- landmark → punto corporal que queremos representar.
- color → color que tendrá el punto.

Antes de dibujarlo se comprueba que la confianza de la detección sea superior a 0.3.

Esto significa que únicamente representamos aquellos puntos cuya detección consideramos suficientemente fiable.

La condición utilizada es:

```
if landmark.confidence > 0.3:
```

Si la confianza es superior a 0.3, se dibuja el punto.

cv2.circle()

Para representar el Landmark utilizamos:

```
cv2.circle(  
img,  
(int(landmark.x), int(landmark.y)),  
6,  
color,  
-1  
)
```

cv2.circle() crea un círculo sobre una imagen.

Sus parámetros indican:

1. img → imagen donde se dibujará.
2. (int(landmark.x), int(landmark.y)) → posición del centro del círculo.
3. 6 → radio del círculo en píxeles.

4. color → color del círculo.

5. -1 → indica que el círculo se dibuja completamente relleno.

Las coordenadas del Landmark pueden ser valores decimales, mientras que OpenCV necesita coordenadas enteras para indicar la posición del píxel.

Por eso utilizamos:

```
int(landmark.x)
```

e:

```
int(landmark.y)
```

Por ejemplo, si el punto tiene:

x = 325.7

y = 241.3

se utilizará aproximadamente:

(325, 241)

para dibujarlo.

draw_connection()

La siguiente función es:

```
def draw_connection(img, p1, p2, color=(255, 255, 255)):
```

Su objetivo es **dibujar una línea entre dos Landmark**.

Recibe:

- img → imagen donde dibujar.
- p1 → primer punto.
- p2 → segundo punto.
- color → color de la línea.

El color predeterminado es (255, 255, 255), que corresponde al color blanco utilizando el formato BGR de OpenCV.

Antes de dibujar la conexión se comprueba que ambos puntos tengan una confianza suficiente:

```
if p1.confidence > 0.3 and p2.confidence > 0.3:
```

Esto evita dibujar una conexión cuando alguno de los dos puntos no ha sido detectado de forma suficientemente fiable.

cv2.line()

La línea se dibuja mediante:

```
cv2.line(  
img,  
(int(p1.x), int(p1.y)),  
(int(p2.x), int(p2.y)),  
color,  
2  
)
```

cv2.line() dibuja una línea entre dos posiciones de la imagen.

En este caso:

- Primer punto → posición de p1.
- Segundo punto → posición de p2.
- color → color de la línea.
- 2 → grosor de la línea.

Por ejemplo, si p1 es el hombro y p2 es el codo, la función dibuja:

Hombro ————— Codo

Esto permite representar visualmente los segmentos del brazo.

draw_angles()

La función draw_angles() es una de las más importantes de este archivo.

Su objetivo es **calcular y mostrar sobre la imagen el ángulo de los dos codos**.

Recibe:

```
def draw_angles(img, pose):
```

- img → imagen sobre la que se escribirá el resultado.
- pose → objeto HumanPose que contiene los puntos corporales.

Esta función utiliza directamente las funciones matemáticas de pose_math.py.

Por tanto, aquí podemos ver claramente cómo diferentes archivos del proyecto empiezan a trabajar juntos.

Cálculo del ángulo del brazo izquierdo

Primero obtenemos el vector correspondiente al brazo izquierdo:

```
brazo_izquierdo = calculate_vector(  
pose.left_elbow,
```

```
pose.left_shoulder  
)
```

Aquí utilizamos el codo como origen y el hombro como destino.

Por tanto, obtenemos un vector que apunta:

Codo → Hombro

Después calculamos el vector correspondiente al antebrazo:

```
antebrazo_izquierdo = calculate_vector(  
pose.left_elbow,  
pose.left_wrist  
)
```

En este caso tenemos:

Codo → Muñeca

Tenemos, por tanto, dos vectores que parten del mismo punto:

Hombro
↑
|
|
Codo —————→ Muñeca

Estos dos vectores forman el ángulo de la articulación del codo.

Cálculo del ángulo

Una vez tenemos ambos vectores:

```
angulo_izquierdo = calculate_angle(  
brazo_izquierdo,  
antebrazo_izquierdo  
)
```

llamamos a `calculate_angle()`, que pertenece a `pose_math.py`.

Esta función realiza todas las operaciones matemáticas que hemos explicado anteriormente:

1. Calcula el producto escalar.
2. Calcula los módulos de los vectores.
3. Obtiene el coseno del ángulo.
4. Aplica el arco coseno.
5. Convierte el resultado a grados.

El resultado se guarda en:

`angulo_izquierdo`

Por ejemplo:

```
angulo_izquierdo = 92.4
```

significaría que el ángulo calculado entre ambos segmentos es aproximadamente de 92,4 grados.

Mostrar el ángulo sobre la imagen

Una vez calculado, utilizamos:

```
cv2.putText()
```

para escribir el valor sobre la imagen.

El texto que se genera es:

```
f"{angulo_izquierdo:.1f}°"
```

Aquí estamos utilizando una cadena de texto formateada.

.1f indica que queremos mostrar el ángulo utilizando un único decimal.

Por ejemplo:

```
92.4378
```

se mostraría como:

```
92.4°
```

La posición del texto se establece cerca del codo:

```
(int(pose.left_elbow.x + 10),  
int(pose.left_elbow.y - 10))
```

Esto significa que el texto se coloca:

- 10 píxeles hacia la derecha del codo.
- 10 píxeles por encima del codo.

De esta manera, el ángulo aparece visualmente asociado a la articulación correspondiente.

Cálculo del brazo derecho

El proceso para el brazo derecho es exactamente el mismo.

Primero calculamos el vector:

```
brazo_derecho = calculate_vector(  
pose.right_elbow,  
pose.right_shoulder  
)
```

que representa:

Codo derecho → Hombro derecho

Después:

```
antebrazo_derecho = calculate_vector(  
pose.right_elbow,  
pose.right_wrist  
)
```

que representa:

Codo derecho → Muñeca derecha

Finalmente calculamos:

```
angulo_derecho = calculate_angle(  
brazo_derecho,  
antebrazo_derecho  
)
```

y obtenemos el ángulo correspondiente al codo derecho.

El resultado también se muestra sobre la imagen mediante `cv2.putText()`.

Bloque de distancias

En el código aparece una sección relacionada con las distancias, pero actualmente está comentada.

Por ejemplo:

```
#def draw_distances(img, pose):
```

y posteriormente todas las instrucciones de esta función también aparecen comentadas.

Esto significa que **esta funcionalidad está preparada en el código, pero actualmente no se está ejecutando**.

La idea de esta función era utilizar `calculate_distance()` de `pose_math.py` para calcular:

- distancia hombro-codo.
- distancia codo-muñeca.

Tanto para el brazo izquierdo como para el derecho.

También estaba preparada la posibilidad de mostrar estas distancias sobre la imagen.

Actualmente no forma parte de la ejecución activa de `pose_drawer.py`, por lo que debemos distinguir entre:

Funcionalidad implementada y activa:

- puntos.
- conexiones.

- ángulos.

Funcionalidad preparada pero desactivada:

- distancias.

Esto es importante para que la documentación represente el estado real del código.

draw_pose()

La última función es:

```
def draw_pose(frame, pose):
```

Esta es la **función principal de pose_drawer.py**.

Su función es reunir todas las operaciones anteriores para generar una representación visual completa de los brazos.

Recibe:

- frame → imagen original procedente de la cámara.
- pose → postura humana detectada.

Copia de la imagen

La primera operación es:

```
frame_draw = frame.copy()
```

Aquí hacemos una copia de la imagen original.

Esto permite realizar todos los dibujos sobre frame_draw sin modificar directamente frame.

Por tanto:

frame → imagen original

frame_draw → copia sobre la que dibujamos

Dibujar los hombros

A continuación se utilizan:

```
draw_landmark(  
frame_draw,  
pose.left_shoulder,  
(0, 0, 255)  
)
```

y:

```
draw_landmark(  
frame_draw,  
pose.right_shoulder,  
(0, 0, 255)  
)
```

Esto dibuja los dos hombros utilizando la función `draw_landmark()`.

El color utilizado es (0, 0, 255).

Como OpenCV utiliza el formato BGR, este valor corresponde al color rojo.

Por tanto, los hombros se representan mediante puntos rojos.

Dibujar los codos

Después se dibujan:

```
pose.left_elbow
```

y:

```
pose.right_elbow
```

utilizando:

```
(0, 255, 0)
```

En formato BGR corresponde al color verde.

Por tanto:

- Hombros → rojo.
- Codos → verde.

Esto permite distinguir visualmente las diferentes articulaciones.

Dibujar las muñecas

Las muñecas se dibujan utilizando:

```
(255, 0, 0)
```

En BGR corresponde al color azul.

Por tanto:

- Hombros → rojo.
- Codos → verde.
- Muñecas → azul.

Esta diferenciación visual permite identificar rápidamente qué punto estamos observando.

Dibujar las conexiones

Después se utilizan cuatro llamadas a `draw_connection()`.

Para el brazo izquierdo:

```
draw_connection(  
frame_draw,  
pose.left_shoulder,  
pose.left_elbow  
)
```

y:

```
draw_connection(  
frame_draw,  
pose.left_elbow,  
pose.left_wrist  
)
```

Esto genera:

Hombro izquierdo — Codo izquierdo — Muñeca izquierda

Para el brazo derecho se realiza exactamente el mismo proceso:

Hombro derecho — Codo derecho — Muñeca derecha

De esta manera, los puntos individuales pasan a formar visualmente la estructura de los brazos.

Dibujar los ángulos

Una vez dibujados los puntos y las conexiones se llama a:

```
draw_angles(  
frame_draw,  
pose  
)
```

Esta función calcula los ángulos de ambos codos y los escribe sobre la imagen.

Es importante observar el flujo:

Landmarks

↓

Vectores

↓

Ángulos

↓

Representación visual

Por tanto, pose_drawer.py utiliza directamente la funcionalidad matemática desarrollada en pose_math.py.

Distancias desactivadas

A continuación aparece:

```
#draw_distances(  
  
frame_draw,  
  
pose  
  
#)
```

Como está comentado, actualmente no se ejecuta.

Si en el futuro se quisiera activar el cálculo y representación de distancias, esta parte permitiría incorporarlo a la representación visual.

Mostrar el resultado

Finalmente:

```
cv2.imshow(  
"KINEMA NEXUS",  
frame_draw  
)
```

cv2.imshow() crea una ventana y muestra en ella la imagen.

El primer parámetro:

"KINEMA NEXUS"

es el nombre que tendrá la ventana.

El segundo:

frame_draw

es la imagen que queremos mostrar.

Por tanto, el resultado final es una imagen de la cámara sobre la que aparecen los elementos que hemos ido añadiendo:

- hombros.
- codos.
- muñecas.
- conexiones entre las articulaciones.
- ángulos de los codos.

Funcionamiento general de pose_drawer.py

Podemos resumir el funcionamiento del archivo de la siguiente manera:

Imagen de cámara



HumanPose proporciona los Landmark



draw_landmark() dibuja los puntos



draw_connection() conecta las articulaciones



pose_math.py calcula los vectores y ángulos



draw_angles() muestra los ángulos



cv2.imshow() muestra el resultado

Por tanto, pose_drawer.py actúa principalmente como **capa de representación visual**. No es el encargado de detectar a la persona ni de realizar por sí mismo las matemáticas de la pose, sino que utiliza las estructuras y funciones creadas anteriormente para mostrar de forma visual lo que el sistema está detectando y calculando.

Esto hace que, hasta este punto, tengamos tres niveles claramente diferenciados:

Landmark

Representa un punto corporal.

HumanPose

Organiza todos los puntos corporales.

pose_math.py

Realiza cálculos matemáticos utilizando esos puntos.

pose_drawer.py

Representa visualmente esos puntos y los resultados de los cálculos.

Este último punto es especialmente importante porque nos permite comprobar de forma visual que las etapas anteriores están funcionando correctamente antes de continuar con las siguientes partes del sistema.

Calibration.py

```
import cv2
import json
import time
import subprocess
import numpy as np
import sys
from pathlib import Path

from ultralytics import YOLO

from pose import HumanPose
from pose_math import calculate_vector, calculate_angle, calculate_distance

# =====
# CONFIGURACIÓN
# =====

CAMERA_INDEX = 0

WINDOW_NAME = "KINEMA NEXUS - Calibration"

REFERENCE_IMAGE = (
    Path(__file__).resolve().parent.parent
    / "images"
    / "postura_referencia.png"
)

PANEL_WIDTH = 640
PANEL_HEIGHT = 480
```

```
MIN_CONFIDENCE = 0.3
```

```
MIN_ANGLE = 50
```

```
MAX_ANGLE = 170
```

```
CALIBRATION_DURATION = 4.0
```

```
SAMPLE_INTERVAL = 0.25
```

```
# =====
```

```
# IMAGEN DE REFERENCIA
```

```
# =====
```

```
def resize_reference_image(image):
```

```
    height, width = image.shape[:2]
```

```
    # Mantener proporción
```

```
    scale = min(
```

```
        PANEL_WIDTH / width,
```

```
        PANEL_HEIGHT / height
```

```
    )
```

```
    new_width = int(width * scale)
```

```
    new_height = int(height * scale)
```

```
    resized = cv2.resize(
```

```
        image,
```

```
        (new_width, new_height)
```

```
    )
```

```
    # Crear panel negro
```

```

panel = np.zeros(
    (PANEL_HEIGHT, PANEL_WIDTH, 3),
    dtype=image.dtype
)

# Centrar imagen
x = (PANEL_WIDTH - new_width) // 2
y = (PANEL_HEIGHT - new_height) // 2

panel[
    y:y + new_height,
    x:x + new_width
] = resized

return panel

# =====
# INICIAR HUMAN POSE DETECTION
# =====

def start_human_pose_detection():

    human_pose_path = (
        Path(__file__).resolve().parent
        / "human_pose_detection.py"
    )

    print("Calibración completada.")
    print("Iniciando Human Pose Detection...")

    subprocess.run(
        [sys.executable, str(human_pose_path)]

```

```
)
```

```
# =====
```

```
# DETECTAR CUERPO SUPERIOR
```

```
# =====
```

```
def detect_upper_body(keypoints):
```

```
    if keypoints is None or len(keypoints.xy) == 0:
```

```
        return False
```

```
    conf = keypoints.conf[0]
```

```
    # Hombros, codos y muñecas
```

```
    required_points = [5, 6, 7, 8, 9, 10]
```

```
    for point in required_points:
```

```
        if conf[point] < MIN_CONFIDENCE:
```

```
            return False
```

```
    return True
```

```
# =====
```

```
# CREAR HUMAN POSE
```

```
# =====
```

```
def create_pose(keypoints):
```

```
    pose = HumanPose()
```

```
puntos = keypoints.xy[0]
```

```
conf = keypoints.conf[0]
```

```
# -----
```

```
# Hombros
```

```
# -----
```

```
pose.left_shoulder.x = float(puntos[5][0])
```

```
pose.left_shoulder.y = float(puntos[5][1])
```

```
pose.left_shoulder.confidence = float(conf[5])
```

```
pose.right_shoulder.x = float(puntos[6][0])
```

```
pose.right_shoulder.y = float(puntos[6][1])
```

```
pose.right_shoulder.confidence = float(conf[6])
```

```
# -----
```

```
# Codos
```

```
# -----
```

```
pose.left_elbow.x = float(puntos[7][0])
```

```
pose.left_elbow.y = float(puntos[7][1])
```

```
pose.left_elbow.confidence = float(conf[7])
```

```
pose.right_elbow.x = float(puntos[8][0])
```

```
pose.right_elbow.y = float(puntos[8][1])
```

```
pose.right_elbow.confidence = float(conf[8])
```

```
# -----
```

```
# Muñecas
```

```
# -----
```

```
pose.left_wrist.x = float(puntos[9][0])
pose.left_wrist.y = float(puntos[9][1])
pose.left_wrist.confidence = float(conf[9])
```

```
pose.right_wrist.x = float(puntos[10][0])
pose.right_wrist.y = float(puntos[10][1])
pose.right_wrist.confidence = float(conf[10])
```

```
return pose
```

```
# =====
# COMPROBAR ÁNGULOS
# =====
```

```
def check_pose_angles(pose):
```

```
# -----
# Brazo izquierdo
# -----
```

```
brazo_izquierdo = calculate_vector(
    pose.left_elbow,
    pose.left_shoulder
)
```

```
antebrazo_izquierdo = calculate_vector(
    pose.left_elbow,
    pose.left_wrist
)
```

```
angulo_izquierdo = calculate_angle(
```

```

    brazo_izquierdo,
    antebrazo_izquierdo
)

# -----
# Brazo derecho
# -----

brazo_derecho = calculate_vector(
    pose.right_elbow,
    pose.right_shoulder
)

antebrazo_derecho = calculate_vector(
    pose.right_elbow,
    pose.right_wrist
)

angulo_derecho = calculate_angle(
    brazo_derecho,
    antebrazo_derecho
)

# -----
# Comprobar rangos
# -----

left_valid = (
    MIN_ANGLE <= angulo_izquierdo <= MAX_ANGLE
)

```



```
right_valid = (  
    MIN_ANGLE <= angulo_derecho <= MAX_ANGLE  
)
```

```
return left_valid and right_valid
```

```
# =====  
# MEDIR DISTANCIAS  
# =====
```

```
def calculate_pose_measurements(pose):
```

```
    return {  
        "left_arm_length": calculate_distance(  
            pose.left_shoulder,  
            pose.left_elbow  
        ),  
  
        "left_forearm_length": calculate_distance(  
            pose.left_elbow,  
            pose.left_wrist  
        ),  
  
        "right_arm_length": calculate_distance(  
            pose.right_shoulder,  
            pose.right_elbow  
        ),  
  
        "right_forearm_length": calculate_distance(  
            pose.right_elbow,  
            pose.right_wrist
```

```
)  
}
```

```
# =====
```

```
# GUARDAR CALIBRACIÓN
```

```
# =====
```

```
def save_calibration(measurements):
```

```
    calibration_file = (  
        Path(__file__).resolve().parent.parent  
        / "data"  
        / "calibration.json"  
    )
```

```
    calibration_file.parent.mkdir(  
        exist_ok=True  
    )
```

```
    with open(  
        calibration_file,  
        "w",  
        encoding="utf-8"  
    ) as file:
```

```
        json.dump(  
            measurements,  
            file,  
            indent=4  
        )
```

```

print(
    f"Calibración guardada en: {calibration_file}"
)

# =====
# DIBUJAR LANDMARKS
# =====

def draw_landmarks(frame, keypoints):

    puntos = keypoints.xy[0]
    conf = keypoints.conf[0]

    for i in range(len(puntos)):

        if conf[i] > MIN_CONFIDENCE:

            x = int(puntos[i][0])
            y = int(puntos[i][1])

            cv2.circle(
                frame,
                (x, y),
                5,
                (0, 255, 0),
                -1
            )

# =====
# MENSAJES
# =====

```

```
def draw_calibration_messages(frame):
```

```
    cv2.putText(
        frame,
        "CALIBRACION INICIAL",
        (20, 35),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.8,
        (0, 255, 255),
        2
    )
```

```
    cv2.putText(
        frame,
        "Colocate dentro del area de la camara",
        (20, 70),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.6,
        (255, 255, 255),
        2
    )
```

```
    cv2.putText(
        frame,
        "Adopta la postura indicada",
        (20, 100),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.6,
        (255, 255, 255),
        2
    )
```

)

=====

CALIBRATION

=====

def start_calibration():

Cargar modelo

model = YOLO("yolo11n-pose.pt")

Cargar imagen de referencia

reference = cv2.imread(
 str(REFERENCE_IMAGE)
)

if reference is None:

print(
 "Error: no se pudo cargar "
 "la imagen de referencia."
)

print(REFERENCE_IMAGE)

```

    return

reference_panel = resize_reference_image(
    reference
)

# -----
# Abrir cámara
# -----

cap = cv2.VideoCapture(CAMERA_INDEX)

if not cap.isOpened():

    print(
        "Error: no se pudo abrir la cámara."
    )

    return

# -----
# Variables de calibración
# -----

calibrating = False

calibration_start_time = None

last_sample_time = None

measurements = {

```

```
"left_arm_length": [],  
"left_forearm_length": [],  
"right_arm_length": [],  
"right_forearm_length": []  
}
```

```
# =====
```

```
# BUCLE PRINCIPAL
```

```
# =====
```

```
while True:
```

```
    ret, frame = cap.read()
```

```
    if not ret:
```

```
        print(  
            "Error: no se pudo obtener el frame."  
        )
```

```
        break
```

```
# -----
```

```
# Inferencia YOLO
```

```
# -----
```

```
results = model(frame)
```

```
keypoints = results[0].keypoints
```

```
# -----
```

```
# Dibujar detección
```

```
# -----
```

```
if (  
    keypoints is not None  
    and len(keypoints.xy) > 0  
):
```

```
    draw_landmarks(  
        frame,  
        keypoints  
    )
```

```
# -----
```

```
# Mensajes generales
```

```
# -----
```

```
draw_calibration_messages(frame)
```

```
# =====
```

```
# DETECCIÓN DE PERSONA
```

```
# =====
```

```
pose_valid = False
```

```
pose = None
```

```
if (  
    keypoints is not None  
    and len(keypoints.xy) > 0  
):
```



```

if detect_upper_body(keypoints):

    # Crear HumanPose
    pose = create_pose(
        keypoints
    )

    # Comprobar postura
    pose_valid = check_pose_angles(
        pose
    )

# =====

# POSTURA CORRECTA

# =====

if pose_valid:

    cv2.putText(
        frame,
        "POSTURA CORRECTA",
        (20, 135),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.6,
        (0, 255, 0),
        2
    )

# -----

# INICIAR CALIBRACIÓN

# -----

```

```
if not calibrating:
```

```
    calibrating = True
```

```
    calibration_start_time = time.time()
```

```
    last_sample_time = calibration_start_time
```

```
    measurements = {  
        "left_arm_length": [],  
        "left_forearm_length": [],  
        "right_arm_length": [],  
        "right_forearm_length": []  
    }
```

```
# -----
```

```
# TIEMPO DE CALIBRACIÓN
```

```
# -----
```

```
current_time = time.time()
```

```
calibration_elapsed = (  
    current_time  
    - calibration_start_time  
)
```

```
# -----
```

```
# TOMAR MUESTRA
```

```
# -----
```

```
if (
    current_time - last_sample_time
    >= SAMPLE_INTERVAL
):

    sample = calculate_pose_measurements(
        pose
    )

    measurements[
        "left_arm_length"
    ].append(
        sample["left_arm_length"]
    )

    measurements[
        "left_forearm_length"
    ].append(
        sample["left_forearm_length"]
    )

    measurements[
        "right_arm_length"
    ].append(
        sample["right_arm_length"]
    )

    measurements[
        "right_forearm_length"
    ].append(
        sample["right_forearm_length"]
    )
```

```

    )

    last_sample_time = current_time

    # -----
    # MOSTRAR PROGRESO
    # -----

    remaining_time = max(
        0,
        CALIBRATION_DURATION
        - calibration_elapsed
    )

    if calibrating:

        cv2.putText(
            frame,
            f"CALIBRANDO: {remaining_time:.1f}s",
            (20, 165),
            cv2.FONT_HERSHEY_SIMPLEX,
            0.6,
            (0, 255, 255),
            2
        )

    # -----
    # FINALIZAR CALIBRACIÓN
    # -----

    if (

```

```
calibrating
and calibration_elapsed >= CALIBRATION_DURATION
):
```

```
if len(
    measurements["left_arm_length"]
) > 0:
```

```
    calibrated_measurements = {
```

```
        "left_arm_length": float(
            np.mean(
                measurements[
                    "left_arm_length"
                ]
            )
        ),
```

```
        "left_forearm_length": float(
            np.mean(
                measurements[
                    "left_forearm_length"
                ]
            )
        ),
```

```
        "right_arm_length": float(
            np.mean(
                measurements[
                    "right_arm_length"
                ]
            )
        ),
```

```

    )
),

"right_forearm_length": float(
    np.mean(
        measurements[
            "right_forearm_length"
        ]
    )
)
}

save_calibration(
    calibrated_measurements
)

print(
    "=====
)

print(
    "CALIBRACION COMPLETADA"
)

print(
    calibrated_measurements
)

print(
    "=====
)

```

```
calibrating = False
```

```
calibration_start_time = None
```

```
last_sample_time = None
```

```
cap.release()
```

```
cv2.destroyAllWindows()
```

```
start_human_pose_detection()
```

```
return
```

```
# =====
```

```
# POSTURA INCORRECTA / PERSONA NO DETECTADA
```

```
# =====
```

```
else:
```

```
if (
```

```
    keypoints is not None
```

```
    and len(keypoints.xy) > 0
```

```
):
```

```
    if detect_upper_body(keypoints):
```

```
        cv2.putText(
```

```
            frame,
```

```
            "AJUSTA LA POSTURA",
```

```
            (20, 135),
```

```
        cv2.FONT_HERSHEY_SIMPLEX,  
        0.6,  
        (0, 0, 255),  
        2  
    )
```

else:

```
    cv2.putText(  
        frame,  
        "COLOCA LOS BRAZOS DENTRO DE LA CAMARA",  
        (20, 135),  
        cv2.FONT_HERSHEY_SIMPLEX,  
        0.6,  
        (0, 0, 255),  
        2  
    )
```

else:

```
    cv2.putText(  
        frame,  
        "NO SE DETECTA NINGUNA PERSONA",  
        (20, 135),  
        cv2.FONT_HERSHEY_SIMPLEX,  
        0.6,  
        (0, 0, 255),  
        2  
    )
```

```
# =====
```



```

# PREPARAR CÁMARA

# =====

camera_panel = cv2.resize(
    frame,
    (PANEL_WIDTH, PANEL_HEIGHT)
)

# =====

# MOSTRAR REFERENCIA + CÁMARA

# =====

combined = cv2.hconcat(
    [
        reference_panel,
        camera_panel
    ]
)

cv2.imshow(
    WINDOW_NAME,
    combined
)

# =====

# SALIR

# =====

if cv2.waitKey(1) & 0xFF == ord("q"):

    break

```

```
# =====
```

```
# LIBERAR RECURSOS
```

```
# =====
```

```
cap.release()
```

```
cv2.destroyAllWindows()
```

```
# =====
```

```
# MAIN
```

```
# =====
```

```
if __name__ == "__main__":
```

```
    start_calibration()
```

Calibration.py explicación

¿Qué función tiene este archivo?

El archivo calibration.py implementa la fase de calibración inicial de Kinema Nexus.

Su objetivo principal es obtener una referencia de las dimensiones de los brazos de la persona que va a utilizar el sistema. Para ello, el programa:

1. Carga el modelo YOLO encargado de detectar la postura humana.
2. Abre la cámara.
3. Detecta los puntos corporales necesarios.
4. Comprueba que los hombros, codos y muñecas sean visibles.
5. Comprueba que la persona mantiene una postura válida.
6. Durante varios segundos recoge medidas de los brazos.
7. Calcula el valor medio de esas medidas.
8. Guarda los resultados en un archivo calibration.json.
9. Una vez terminada la calibración, inicia el sistema principal de detección de postura.

Por tanto, calibration.py actúa como una etapa intermedia entre la detección inicial de la persona y el funcionamiento normal del sistema.

IMPORTACIONES

```
import cv2
```

Importa OpenCV, que se utiliza para trabajar con la cámara, modificar imágenes y mostrar información sobre ellas.

En este archivo se utilizan varias funciones de OpenCV, entre ellas:

- cv2.imread() para cargar la imagen de referencia.
- cv2.resize() para cambiar el tamaño de imágenes.
- cv2.VideoCapture() para acceder a la cámara.
- cv2.circle() para dibujar los puntos detectados.
- cv2.putText() para escribir mensajes sobre la imagen.
- cv2.hconcat() para colocar dos imágenes una al lado de otra.
- cv2.imshow() para mostrar el resultado.
- cv2.waitKey() para detectar la pulsación de teclas.
- cv2.destroyAllWindows() para cerrar las ventanas de OpenCV.

```
import json
```

La librería json permite trabajar con archivos en formato JSON.

En este archivo se utiliza para guardar las medidas obtenidas durante la calibración en `calibration.json`.

JSON es un formato de almacenamiento de información estructurada que permite guardar datos de forma organizada y fácil de leer tanto para personas como para programas.

```
import time
```

La librería `time` permite trabajar con el tiempo.

En este archivo se utiliza principalmente `time.time()` para medir cuánto tiempo ha transcurrido desde el comienzo de la calibración y para controlar cada cuánto se toma una muestra.

```
import subprocess
```

Esta librería permite ejecutar otro programa desde Python.

En Kinema Nexus se utiliza para iniciar el archivo `human_pose_detection.py` cuando la calibración ha terminado.

```
import numpy as np
```

NumPy es una librería utilizada para operaciones matemáticas y tratamiento de datos numéricos.

En este archivo se utiliza concretamente `np.mean()` para calcular la media de las medidas obtenidas durante la calibración.

```
import sys
```

La librería `sys` permite acceder a información y elementos relacionados con el propio intérprete de Python.

Aquí se utiliza `sys.executable` para obtener la ruta del intérprete de Python que está ejecutando actualmente el programa.

Esto permite iniciar el siguiente archivo utilizando el mismo Python con el que se está ejecutando Kinema Nexus.

```
from pathlib import Path
```

`Path` permite trabajar con rutas de archivos y carpetas de una manera más segura y organizada que construyendo las rutas manualmente como cadenas de texto.

En este archivo se utiliza para localizar:

- La imagen de referencia.
- El archivo de calibración.
- El archivo `human_pose_detection.py`.

```
from ultralytics import YOLO
```

Aquí se importa `YOLO` desde `Ultralytics`.

`YOLO` es el modelo de inteligencia artificial utilizado por Kinema Nexus para detectar la postura humana y obtener sus puntos corporales.

En este archivo se utiliza para cargar yolo11n-pose.pt.

Este modelo proporciona, entre otros datos, las coordenadas de los puntos corporales y la confianza de cada detección.

```
from pose import HumanPose
```

Importa la clase HumanPose, explicada anteriormente.

Esto permite transformar los puntos detectados por YOLO en la estructura de datos propia de Kinema Nexus.

También se importan:

- calculate_vector
- calculate_angle
- calculate_distance

Estas funciones pertenecen a pose_math.py y permiten realizar las operaciones matemáticas necesarias durante la calibración.

CONFIGURACIÓN

CAMERA_INDEX = 0

Indica qué cámara debe utilizar OpenCV.

El valor 0 normalmente corresponde a la cámara principal del ordenador.

WINDOW_NAME = "KINEMA NEXUS - Calibration"

Es el nombre que tendrá la ventana de OpenCV utilizada durante la calibración.

REFERENCE_IMAGE

La imagen de referencia se obtiene mediante:

```
Path(__file__).resolve().parent.parent / "images" / "postura_referencia.png"
```

Esta construcción busca la imagen postura_referencia.png dentro de la carpeta images del proyecto.

Aquí aparecen varios elementos importantes de Path.

`__file__`

Representa la ubicación del archivo Python que se está ejecutando.

`resolve()`

Obtiene la ruta absoluta.

`parent`

Permite acceder a la carpeta que contiene esa ruta.

De esta manera, el programa puede localizar la imagen independientemente de desde qué carpeta se haya ejecutado el programa.

PANEL_WIDTH = 640

PANEL_HEIGHT = 480

Definen el tamaño de los paneles que se mostrarán en pantalla.

Cada panel tendrá 640 píxeles de ancho y 480 píxeles de alto.

MIN_CONFIDENCE = 0.3

Es el umbral mínimo de confianza utilizado para aceptar un punto corporal.

Si YOLO considera que un punto tiene una confianza inferior a 0.3, ese punto no se considera suficientemente fiable para esta fase.

MIN_ANGLE = 50

MAX_ANGLE = 170

Estos valores definen el rango de ángulos considerado válido durante la postura de calibración.

Por tanto, el ángulo de cada codo debe encontrarse entre 50° y 170°.

CALIBRATION_DURATION = 4.0

Indica que la fase de recogida de medidas dura cuatro segundos.

SAMPLE_INTERVAL = 0.25

Indica que se intenta tomar una nueva muestra cada 0,25 segundos.

Por tanto, durante los cuatro segundos se pueden obtener aproximadamente 16 muestras, siempre que la detección sea válida y el procesamiento se mantenga dentro de los tiempos previstos.

IMAGEN DE REFERENCIA

resize_reference_image()

La función `resize_reference_image(image)` se encarga de adaptar la imagen de referencia al tamaño del panel utilizado por el programa.

Recibe:

image

La imagen que se quiere adaptar.

Primero obtiene sus dimensiones:

`height, width = image.shape[:2]`

En OpenCV una imagen se representa como una matriz de datos.

shape proporciona sus dimensiones.

En una imagen en color normalmente tenemos:

alto × ancho × canales

Por eso, `image.shape[:2]` selecciona únicamente las dos primeras dimensiones:

- altura.
- anchura.

A continuación se calcula:

```
scale = min(PANEL_WIDTH / width, PANEL_HEIGHT / height)
```

Este cálculo determina cuánto se puede reducir o ampliar la imagen para que quepa completamente dentro del panel.

Se utiliza `min()` porque deben cumplirse simultáneamente las dos restricciones:

- que no supere el ancho disponible.
- que no supere el alto disponible.

De esta manera se conserva la proporción original de la imagen.

Después se calculan las nuevas dimensiones:

```
new_width = int(width * scale)
```

```
new_height = int(height * scale)
```

El factor `scale` se aplica tanto al ancho como al alto.

Esto es importante porque permite cambiar el tamaño sin deformar la imagen.

A continuación:

```
resized = cv2.resize(image, (new_width, new_height))
```

`cv2.resize()` cambia el tamaño de una imagen.

Recibe:

- la imagen original.
- el nuevo tamaño.

El resultado es la imagen adaptada.

Después se crea un panel negro:

```
panel = np.zeros((PANEL_HEIGHT, PANEL_WIDTH, 3), dtype=image.dtype)
```

`np.zeros()` crea una matriz cuyos valores iniciales son cero.

En una imagen BGR, el valor (0, 0, 0) corresponde al negro.

Los tres canales representan:

- B: azul.
- G: verde.
- R: rojo.

Por tanto, esta instrucción crea una imagen negra de 640 × 480 píxeles.

```
dtype=image.dtype
```

hace que el panel utilice el mismo tipo de datos que la imagen original.

Posteriormente se calcula la posición donde debe colocarse la imagen:

```
x = (PANEL_WIDTH - new_width) // 2
```

```
y = (PANEL_HEIGHT - new_height) // 2
```

Estos cálculos permiten centrar la imagen dentro del panel.

Finalmente:

```
panel[y:y + new_height, x:x + new_width] = resized
```

coloca la imagen redimensionada dentro del panel negro.

La función devuelve:

```
return panel
```

Por tanto, el resultado final es una imagen de tamaño fijo de 640 × 480 que contiene la imagen de referencia centrada y sin deformarla.

INICIAR HUMAN POSE DETECTION

```
start_human_pose_detection()
```

Esta función se encarga de iniciar la siguiente etapa del sistema una vez terminada la calibración.

Primero construye la ruta:

```
human_pose_path = Path(__file__).resolve().parent / "human_pose_detection.py"
```

La intención es localizar el archivo human_pose_detection.py dentro de la misma carpeta que calibration.py.

Después muestra:

```
"Calibración completada."
```

```
"Iniciando Human Pose Detection..."
```

Finalmente utiliza:

```
subprocess.run([sys.executable, str(human_pose_path)])
```

Aquí se ejecuta otro programa Python desde el programa actual.

sys.executable proporciona la ruta del intérprete de Python que está ejecutando actualmente Kinema Nexus.

str(human_pose_path) convierte el objeto Path en una cadena de texto que representa la ruta del archivo.

Por tanto, esta instrucción equivale conceptualmente a ejecutar:

Python → human_pose_detection.py

Esta función establece así la transición entre la calibración y el funcionamiento normal del sistema.

DETECTAR CUERPO SUPERIOR

`detect_upper_body()`

La función `detect_upper_body(keypoints)` comprueba si los puntos necesarios de la parte superior del cuerpo han sido detectados correctamente.

Recibe:

`keypoints`

Es la información de puntos corporales proporcionada por YOLO.

Primero comprueba si no existen puntos detectados.

Si no hay información suficiente, devuelve `False`.

Después:

`conf = keypoints.conf[0]`

obtiene las confianzas correspondientes a la primera persona detectada.

El sistema necesita concretamente:

`required_points = [5, 6, 7, 8, 9, 10]`

Estos índices corresponden a:

5 → hombro izquierdo

6 → hombro derecho

7 → codo izquierdo

8 → codo derecho

9 → muñeca izquierda

10 → muñeca derecha

Estos son precisamente los seis puntos necesarios para analizar los dos brazos.

Después se comprueba cada punto.

Si cualquiera de ellos tiene una confianza inferior a `MIN_CONFIDENCE`, la función devuelve `False`.

Si todos los puntos cumplen el mínimo:

`return True`

Por tanto, esta función responde a una pregunta muy concreta:

"¿Tenemos detectados con suficiente confianza todos los puntos necesarios para trabajar con los brazos?"

CREAR HUMAN POSE

`create_pose()`

La función `create_pose(keypoints)` convierte la información proporcionada por YOLO en un objeto `HumanPose`.

Primero:

```
pose = HumanPose()
```

crea una estructura `HumanPose` nueva.

Después:

```
puntos = keypoints.xy[0]
```

obtiene las coordenadas de los puntos de la primera persona detectada.

Y:

```
conf = keypoints.conf[0]
```

obtiene las confianzas correspondientes.

A partir de ahí, cada punto de YOLO se copia al Landmark correspondiente.

Por ejemplo:

```
pose.left_shoulder.x = float(puntos[5][0])
```

```
pose.left_shoulder.y = float(puntos[5][1])
```

```
pose.left_shoulder.confidence = float(conf[5])
```

Aquí se está haciendo la conversión:

YOLO → Landmark → HumanPose

El índice 5 representa el hombro izquierdo.

Se guardan:

- coordenada X.
- coordenada Y.
- confianza.

Lo mismo se hace para:

Hombro derecho → índice 6

Codo izquierdo → índice 7

Codo derecho → índice 8

Muñeca izquierda → índice 9

Muñeca derecha → índice 10

La función termina devolviendo:

return pose

Por tanto, convierte el formato propio de YOLO en la estructura interna que utiliza Kinema Nexus.

Esta función es especialmente importante porque actúa como puente entre la inteligencia artificial y el resto del proyecto.

COMPROBAR ÁNGULOS

check_pose_angles()

Esta función comprueba si los dos brazos se encuentran dentro del rango angular definido para la calibración.

Primero se calcula el vector desde el codo izquierdo hacia el hombro:

calculate_vector(pose.left_elbow, pose.left_shoulder)

Después se calcula el vector desde el codo izquierdo hacia la muñeca:

calculate_vector(pose.left_elbow, pose.left_wrist)

Ambos vectores tienen como origen el codo.

Esto permite calcular correctamente el ángulo de la articulación.

Después:

calculate_angle(brazo_izquierdo, antebrazo_izquierdo)

calcula el ángulo entre ambos vectores.

El mismo procedimiento se realiza para el brazo derecho.

Después se comprueba:

MIN_ANGLE <= angulo_izquierdo <= MAX_ANGLE

y:

MIN_ANGLE <= angulo_derecho <= MAX_ANGLE

Es decir, ambos ángulos deben encontrarse entre 50° y 170°.

Finalmente:

return left_valid and right_valid

La función solamente devuelve True si los dos brazos cumplen simultáneamente las condiciones.

Esto evita comenzar una calibración cuando uno de los brazos está en una postura incorrecta.

MEDIR DISTANCIAS

`calculate_pose_measurements()`

Esta función obtiene las cuatro medidas que interesan durante la calibración.

Devuelve un diccionario con:

`left_arm_length`

Distancia entre hombro izquierdo y codo izquierdo.

`left_forearm_length`

Distancia entre codo izquierdo y muñeca izquierda.

`right_arm_length`

Distancia entre hombro derecho y codo derecho.

`right_forearm_length`

Distancia entre codo derecho y muñeca derecha.

Para calcularlas utiliza `calculate_distance()` de `pose_math.py`.

Por ejemplo:

`calculate_distance(pose.left_shoulder, pose.left_elbow)`

calcula la distancia en píxeles entre ambos puntos.

Es importante destacar que estas medidas todavía son medidas en el espacio de la imagen.

Es decir, representan distancias en píxeles y no directamente centímetros o milímetros.

La función devuelve todas las medidas organizadas en un diccionario.

GUARDAR CALIBRACIÓN

`save_calibration()`

Esta función guarda las medidas finales de calibración.

Primero se construye la ruta:

`data/calibration.json`

mediante `Path`.

Después:

`calibration_file.parent.mkdir(exist_ok=True)`

se asegura de que exista la carpeta `data`.

`mkdir()` crea la carpeta.

`exist_ok=True` evita generar un error si la carpeta ya existe.

Después se abre el archivo:

with `open(calibration_file, "w", encoding="utf-8")` as `file`:

La "w" significa que el archivo se abre en modo escritura.

A continuación:

```
json.dump(measurements, file, indent=4)
```

escribe los datos en formato JSON.

measurements contiene las medidas obtenidas.

file indica dónde escribirlas.

indent=4 hace que el archivo tenga una estructura visualmente organizada con sangrías de cuatro espacios.

Finalmente se muestra por consola dónde se ha guardado.

DIBUJAR LANDMARKS

```
draw_landmarks()
```

Esta función dibuja los puntos detectados por YOLO sobre la imagen de la cámara.

Recibe:

frame

La imagen sobre la que se dibujará.

keypoints

Los puntos detectados por YOLO.

Primero obtiene:

```
puntos = keypoints.xy[0]
```

y:

```
conf = keypoints.conf[0]
```

Después recorre todos los puntos detectados.

Para cada uno comprueba:

```
if conf[i] > MIN_CONFIDENCE
```

Solo dibuja aquellos cuya confianza sea superior a 0.3.

Las coordenadas se convierten a enteros:

```
x = int(puntos[i][0])
```

```
y = int(puntos[i][1])
```

Esto es necesario porque los píxeles de la imagen se representan mediante posiciones enteras.

Después:

```
cv2.circle(frame, (x, y), 5, (0, 255, 0), -1)
```

dibuja un círculo.

Los argumentos significan:

- frame: imagen sobre la que dibujar.
- (x, y): posición del punto.
- 5: radio del círculo.
- (0, 255, 0): color verde en formato BGR.
- -1: círculo relleno.

Esta función permite visualizar directamente los puntos que YOLO está detectando.

MENSAJES

`draw_calibration_messages()`

Esta función añade los mensajes informativos que aparecen durante la calibración.

Utiliza:

`cv2.putText()`

para escribir texto sobre la imagen.

Se muestran tres mensajes principales:

"CALIBRACION INICIAL"

"Colocate dentro del area de la camara"

"Adopta la postura indicada"

En cada llamada a `cv2.putText()` se indican:

- imagen donde escribir.
- texto.
- posición.
- tipo de fuente.
- tamaño.
- color.
- grosor.

Por ejemplo:

`cv2.FONT_HERSHEY_SIMPLEX`

es una fuente incluida en OpenCV.

Los colores están expresados en formato BGR.

CALIBRATION

`start_calibration()`

Esta es la función principal de todo el archivo.

Es la encargada de ejecutar la secuencia completa de calibración.

Cargar el modelo

La función comienza con:

```
model = YOLO("yolo11n-pose.pt")
```

Aquí se crea el objeto que representa el modelo YOLO de detección de postura.

El archivo `yolo11n-pose.pt` contiene el modelo entrenado que se utilizará para detectar los puntos corporales.

Una vez cargado, `model` puede recibir imágenes y producir detecciones.

Cargar la imagen de referencia

Se utiliza:

```
reference = cv2.imread(str(REFERENCE_IMAGE))
```

`cv2.imread()` carga una imagen desde el disco.

`str(REFERENCE_IMAGE)` convierte la ruta construida con `Path` en texto.

Después se comprueba si la imagen se ha cargado correctamente.

Si OpenCV no ha podido cargarla, se muestra un mensaje de error y la función termina.

Después se prepara la imagen mediante:

```
reference_panel = resize_reference_image(reference)
```

El resultado será un panel de 640×480 píxeles.

Abrir la cámara

Se utiliza:

```
cap = cv2.VideoCapture(CAMERA_INDEX)
```

`cv2.VideoCapture()` crea una conexión con la cámara indicada.

Como `CAMERA_INDEX = 0`, se intenta utilizar la cámara principal.

Después:

```
cap.isOpened()
```

comprueba si la cámara se ha abierto correctamente.

Si no es así, se muestra un error y termina la función.

Variables de calibración

Se crean:

`calibrating = False`

Indica si actualmente se está realizando la recogida de medidas.

`calibration_start_time = None`

Guardará el momento en que comienza la calibración.

`last_sample_time = None`

Guardará el momento en que se tomó la última muestra.

También se crea `measurements`, un diccionario que contiene cuatro listas.

Cada lista almacenará las diferentes medidas obtenidas durante los cuatro segundos:

- longitud del brazo izquierdo.
- longitud del antebrazo izquierdo.
- longitud del brazo derecho.
- longitud del antebrazo derecho.

BUCLE PRINCIPAL DE CALIBRACIÓN

El programa entra entonces en:

`while True`

Este es el ciclo que mantiene activa la cámara y procesa continuamente los frames.

Capturar un frame

Se utiliza:

`ret, frame = cap.read()`

`cap.read()` intenta obtener una nueva imagen de la cámara.

Devuelve dos elementos:

`ret`

Indica si la captura se ha realizado correctamente.

`frame`

Contiene la imagen capturada.

Si `ret` es falso, el programa muestra un error y sale del bucle.

Realizar la inferencia YOLO

Se ejecuta:

`results = model(frame)`

Aquí se entrega el frame actual al modelo YOLO.

YOLO analiza la imagen y devuelve los resultados de la detección.

Después:

```
keypoints = results[0].keypoints
```

extrae los puntos corporales detectados en el primer resultado.

A partir de aquí, el programa ya dispone de:

- coordenadas de los puntos.
- confianza de cada punto.

Dibujar los puntos detectados

Si existen keypoints válidos:

```
draw_landmarks(frame, keypoints)
```

se dibujan los puntos sobre el frame.

Esto permite al usuario visualizar en tiempo real lo que está detectando el modelo.

Mostrar mensajes generales

Después se ejecuta:

```
draw_calibration_messages(frame)
```

para mostrar las instrucciones de calibración.

COMPROBAR SI EXISTE UNA POSTURA VÁLIDA

En cada frame se empieza suponiendo:

```
pose_valid = False
```

```
pose = None
```

Si existe una detección, se llama a:

```
detect_upper_body(keypoints)
```

Si los seis puntos necesarios están correctamente detectados, se crea un HumanPose:

```
pose = create_pose(keypoints)
```

Después se comprueba la postura:

```
pose_valid = check_pose_angles(pose)
```

Por tanto, para considerar válida la postura deben cumplirse dos condiciones:

1. Los seis puntos necesarios tienen suficiente confianza.
2. Los dos codos se encuentran dentro del rango angular permitido.

CUANDO LA POSTURA ES CORRECTA

Si pose_valid es verdadero, se muestra:

"POSTURA CORRECTA"

Después comienza la lógica de calibración.

Inicio de la calibración

Si todavía no se estaba calibrando:

if not calibrating

se establece:

calibrating = True

Después se guarda el momento inicial:

calibration_start_time = time.time()

Y también:

last_sample_time = calibration_start_time

Esto establece el punto de referencia temporal para las mediciones.

Además, las listas de medidas se reinician.

De esta manera, cada nueva calibración comienza desde cero.

Control del tiempo de calibración

En cada iteración se obtiene:

current_time = time.time()

Después:

calibration_elapsed = current_time - calibration_start_time

Esto calcula cuánto tiempo ha pasado desde el inicio.

Por ejemplo, si han pasado dos segundos:

calibration_elapsed = 2

Han transcurrido dos segundos desde el comienzo de la calibración.

Tomar muestras

El programa comprueba:

current_time - last_sample_time >= SAMPLE_INTERVAL

Como SAMPLE_INTERVAL = 0.25, se toma aproximadamente una nueva muestra cada cuarto de segundo.

Cuando corresponde tomar una muestra:

sample = calculate_pose_measurements(pose)

calcula las cuatro distancias.

Después cada medida se añade a su lista correspondiente mediante append().

Finalmente:

```
last_sample_time = current_time
```

actualiza el instante de la última muestra.

Mostrar el progreso

Se calcula:

```
remaining_time = max(0, CALIBRATION_DURATION - calibration_elapsed)
```

Esto obtiene el tiempo que queda.

`max(0, ...)` evita que aparezca un valor negativo cuando el tiempo haya terminado.

Después se muestra:

```
"CALIBRANDO: X.Xs"
```

Por ejemplo:

```
"CALIBRANDO: 2.7s"
```

Esto permite al usuario saber cuánto queda de proceso.

Finalizar la calibración

La calibración termina cuando:

```
calibration_elapsed >= CALIBRATION_DURATION
```

Es decir, cuando han transcurrido al menos cuatro segundos.

Antes de calcular el resultado se comprueba que exista al menos una muestra.

Durante los cuatro segundos se han recogido múltiples medidas.

No se utiliza directamente una sola medición.

En su lugar se calcula la media mediante:

```
np.mean()
```

Por ejemplo:

```
np.mean(measurements["left_arm_length"])
```

calcula la media de todas las medidas del brazo izquierdo.

Esto se hace para los cuatro valores.

El resultado se convierte a float para obtener un número decimal estándar de Python.

La estructura final es `calibrated_measurements`, que contiene las cuatro medidas medias.

El uso de la media tiene como finalidad obtener una referencia más estable que una única detección, reduciendo la influencia de pequeñas variaciones entre frames.

Guardar la calibración

Una vez calculados los valores:

```
save_calibration(calibrated_measurements)
```

guarda los resultados en:

```
data/calibration.json
```

Este archivo pasa a contener la referencia obtenida durante la calibración.

Liberar la cámara y pasar al siguiente módulo

Después de guardar la calibración se restablecen las variables relacionadas con el proceso.

Se libera la cámara mediante:

```
cap.release()
```

Esto desconecta la cámara.

Después:

```
cv2.destroyAllWindows()
```

cierra las ventanas abiertas por OpenCV.

Finalmente:

```
start_human_pose_detection()
```

inicia el archivo:

```
human_pose_detection.py
```

Por tanto, la calibración no termina simplemente cerrando el programa.

Termina realizando una transición automática hacia la siguiente etapa de Kinema Nexus.

CUANDO LA POSTURA ES INCORRECTA

Si pose_valid es falso, el programa diferencia entre varias situaciones.

Si existe una persona detectada y los seis puntos necesarios están disponibles, pero los ángulos no son correctos:

"AJUSTA LA POSTURA"

Si la persona existe pero no se han detectado correctamente todos los puntos de los brazos:

"COLOCA LOS BRAZOS DENTRO DE LA CAMARA"

Y si directamente no hay ninguna detección:

"NO SE DETECTA NINGUNA PERSONA"

Esto permite distinguir entre:

- ausencia de persona.
- persona presente pero brazos no visibles.

- persona correctamente detectada pero postura incorrecta.

PREPARAR EL PANEL DE CÁMARA

Después de procesar el frame:

```
camera_panel = cv2.resize(frame, (PANEL_WIDTH, PANEL_HEIGHT))
```

se cambia el tamaño del frame de la cámara a 640×480 .

De esta forma tendrá exactamente las mismas dimensiones que el panel de referencia.

COMBINAR REFERENCIA Y CÁMARA

Se utiliza:

```
cv2.hconcat([reference_panel, camera_panel])
```

`cv2.hconcat()` concatena imágenes horizontalmente.

En este caso se colocan:

Imagen de referencia | Imagen de la cámara

El resultado es una única imagen formada por dos paneles de 640×480 .

Por tanto, la ventana final tendrá aproximadamente 1280 píxeles de ancho y 480 píxeles de alto.

Esto permite que el usuario vea simultáneamente:

- la postura que debe adoptar.
- su propia postura detectada por la cámara.

MOSTRAR EL RESULTADO

Finalmente:

```
cv2.imshow(WINDOW_NAME, combined)
```

muestra la imagen combinada en una ventana llamada:

"KINEMA NEXUS - Calibration"

Esta ventana se actualiza continuamente dentro del bucle.

SALIR DE LA CALIBRACIÓN MANUALMENTE

El programa comprueba:

```
cv2.waitKey(1) & 0xFF == ord("q")
```

`cv2.waitKey(1)` espera aproximadamente 1 milisegundo por una tecla.

`ord("q")` obtiene el código correspondiente a la tecla q.

Por tanto, si el usuario pulsa q, se rompe el bucle de calibración.

Después se liberan los recursos:

```
cap.release()
```

y:

```
cv2.destroyAllWindows()
```

LIBERAR RECURSOS

Cuando el bucle termina, el programa vuelve a liberar la cámara:

```
cap.release()
```

y cierra las ventanas:

```
cv2.destroyAllWindows()
```

Esto es importante para evitar que la cámara quede ocupada por el programa una vez finalizada la ejecución.

MAIN

Al final aparece:

```
if __name__ == "__main__":
```

```
    start_calibration()
```

Esto significa que `start_calibration()` se ejecutará cuando `calibration.py` se ejecute directamente.

Es el punto de entrada del programa de calibración.

PUNTO DE ENTRADA DEL ARCHIVO

Cuando se ejecuta `calibration.py` directamente, Python llega a la condición:

```
if __name__ == "__main__":
```

y ejecuta:

```
    start_calibration()
```

De esta forma comienza todo el proceso de calibración.

FLUJO COMPLETO DE CALIBRATION.PY

El funcionamiento completo puede entenderse como la siguiente cadena:

Inicio

↓

Cargar YOLO

↓

Cargar imagen de referencia

↓

Abrir cámara

↓

Capturar frame

↓

YOLO detecta puntos corporales

↓

Comprobar hombros, codos y muñecas

↓

Crear HumanPose

↓

Calcular ángulos de los codos

↓

¿Postura correcta?

→ No → Mostrar mensaje y continuar detectando

→ Sí → Comenzar calibración

↓

Recoger muestras durante 4 segundos

↓

Calcular distancias de brazos y antebrazos

↓

Guardar múltiples muestras

↓

Calcular media de cada medida

↓

Guardar calibration.json

↓

Liberar cámara

↓

Iniciar human_pose_detection.py

RELACIÓN CON LOS ARCHIVOS ANTERIORES

calibration.py es el primer archivo que integra de forma clara varias de las estructuras que se han explicado anteriormente.

La relación es:

Landmark

Representa cada punto corporal individual.



HumanPose

Agrupar los puntos corporales y proporciona una estructura organizada del cuerpo.



pose_math.py

Utiliza esos puntos para calcular:

- vectores.
- ángulos.
- distancias.



calibration.py

Utiliza esas herramientas para determinar una referencia de las dimensiones de los brazos del usuario.

Además, calibration.py introduce dos elementos nuevos importantes:

YOLO

Es el sistema que obtiene los puntos corporales a partir de la imagen.

JSON

Es el sistema utilizado para almacenar los resultados de la calibración.

Por tanto, este archivo empieza a conectar la detección de inteligencia artificial con los datos que posteriormente utilizará el sistema de control.

PAPEL DE LA CALIBRACIÓN DENTRO DE KINEMA NEXUS

La calibración tiene una función fundamental: adaptar el sistema a la persona que está utilizando el dispositivo.

La detección de postura proporciona coordenadas en la imagen.

Sin embargo, cada persona puede tener unas proporciones corporales diferentes.

Por ejemplo, dos personas pueden realizar una postura aparentemente idéntica pero tener diferentes distancias entre:

- hombro y codo.
- codo y muñeca.

La calibración obtiene estas distancias como referencia.

De esta manera, las etapas posteriores pueden trabajar teniendo en cuenta las proporciones del usuario en lugar de asumir unas dimensiones corporales universales.

Es importante recordar que las medidas obtenidas en esta fase siguen estando expresadas en píxeles. Este archivo no realiza todavía una conversión directa de píxeles a milímetros.

RESUMEN DEL ARCHIVO

calibration.py es el módulo encargado de realizar la calibración inicial de Kinema Nexus.

Sus funciones principales son:

- Cargar el modelo YOLO de pose.
- Capturar imágenes de la cámara.
- Detectar los puntos corporales.
- Verificar que los puntos necesarios tengan suficiente confianza.
- Verificar que la postura sea válida.
- Crear objetos HumanPose.
- Calcular los ángulos de los codos.
- Medir las longitudes aparentes de brazos y antebrazos.
- Recoger varias muestras.
- Calcular valores medios.
- Guardar los resultados en calibration.json.
- Mostrar al usuario el estado de la calibración.
- Pasar automáticamente al módulo human_pose_detection.py.

Con este archivo, Kinema Nexus deja de ser únicamente un sistema capaz de detectar puntos y calcular ángulos y empieza a incorporar una referencia individual del usuario que podrá ser utilizada por las siguientes etapas del proyecto.

Relative_pose.py

```
from pose_math import calculate_vector
```

```
# =====
```

```
# POSICIONES RELATIVAS DEL BRAZO
```

```
# =====
```

```
def calculate_relative_arm_pose(pose):
```

```
    """
```

```
    Calcula los vectores relativos de los segmentos  
    de ambos brazos del operador.
```

```
    Brazo:
```

```
        Hombro → Codo
```

```
    Antebrazo:
```

```
        Codo → Muñeca
```

```
    """
```

```
# -----
```

```
# Brazo izquierdo
```

```
# -----
```

```
left_arm = calculate_vector(
```

```
    pose.left_shoulder,
```

```
    pose.left_elbow
```

```
)
```

```
left_forearm = calculate_vector(
```

```
    pose.left_elbow,
```

```
    pose.left_wrist
```

```

)

# -----
# Brazo derecho
# -----

right_arm = calculate_vector(
    pose.right_shoulder,
    pose.right_elbow
)

right_forearm = calculate_vector(
    pose.right_elbow,
    pose.right_wrist
)

# -----
# Devolver vectores relativos
# -----

return {
    "left_arm": {
        "upper_arm": left_arm,
        "forearm": left_forearm
    },

    "right_arm": {
        "upper_arm": right_arm,
        "forearm": right_forearm
    }
}

```

Relative_pose.py Explicación

Importación de la función matemática

```
from pose_math import calculate_vector
```

Esta línea importa la función `calculate_vector` desde el archivo `pose_math.py`.

La función `calculate_vector` es la encargada de calcular el **vector que existe entre dos puntos corporales**.

Por ejemplo, si tenemos:

Hombro → Codo

podemos calcular un vector que indique:

- cuánto se ha desplazado el brazo en X;
- cuánto se ha desplazado el brazo en Y;
- y, por tanto, en qué dirección está orientado el segmento.

Esto permite separar las responsabilidades entre diferentes archivos:

`pose.py`



Contiene los landmarks de la persona

`pose_math.py`



Realiza los cálculos matemáticos

`relative_pose.py`



Utiliza esos cálculos para obtener

la posición relativa de los brazos

De esta forma, `relative_pose.py` no necesita implementar de nuevo las operaciones matemáticas necesarias para calcular los vectores.

Posiciones relativas del brazo

```
# =====  
  
# POSICIONES RELATIVAS DEL BRAZO  
  
# =====
```

Este comentario indica la finalidad principal del bloque de código.

La función que aparece a continuación calcula la **posición relativa de los segmentos de ambos brazos**.

La palabra *relativa* es importante.

En lugar de trabajar únicamente con las coordenadas absolutas de cada landmark:

Hombro → (x, y)

Codo → (x, y)

Muñeca → (x, y)

calculamos la relación existente entre ellos:

Hombro → Codo

Codo → Muñeca

De esta manera obtenemos información sobre la **orientación de cada segmento del brazo**, independientemente de la posición global de la persona dentro de la imagen.

Función `calculate_relative_arm_pose`

```
def calculate_relative_arm_pose(pose):
```

Aquí definimos la función `calculate_relative_arm_pose`.

Su objetivo es recibir una pose y calcular los vectores correspondientes a los diferentes segmentos de ambos brazos.

El parámetro:

`pose`

representa el objeto `HumanPose` que contiene los diferentes landmarks corporales.

Por tanto, dentro de esta función podemos acceder, por ejemplo, a:

`pose.left_shoulder`

`pose.left_elbow`

`pose.left_wrist`

y a los equivalentes del brazo derecho.

La función no necesita recibir cada landmark por separado porque todos ellos ya están agrupados dentro del objeto `pose`.

Documentación de la función

```
"""
```

Calcula los vectores relativos de los segmentos

de ambos brazos del operador.

Brazo:

Hombro → Codo

Antebrazo:

Codo → Muñeca

"""

Este bloque es un **docstring**.

Un docstring es una cadena de texto que se coloca normalmente al principio de una función para explicar qué hace y qué información maneja.

En este caso se especifica que la función calcula los vectores relativos de ambos brazos.

Se distinguen dos segmentos principales:

Brazo:

Hombro → Codo

y:

Antebrazo:

Codo → Muñeca

Esto es importante porque el brazo completo se puede dividir en diferentes segmentos articulados.

La estructura utilizada es:

Hombro

↓

Codo

↓

Muñeca

Por tanto:

Hombro → Codo = brazo superior

Codo → Muñeca = antebrazo

Esta representación será especialmente útil posteriormente para analizar la postura y el movimiento de las extremidades.

Brazo izquierdo

Brazo izquierdo

Este bloque comienza el cálculo correspondiente al brazo izquierdo.

Primero calculamos el vector del brazo superior.

Vector del brazo izquierdo

```
left_arm = calculate_vector(  
    pose.left_shoulder,  
    pose.left_elbow  
)
```

Aquí utilizamos `calculate_vector` para obtener el vector que va desde el **hombro izquierdo hasta el codo izquierdo**.

Los dos argumentos son:

`pose.left_shoulder`

que representa el landmark del hombro izquierdo,

y:

`pose.left_elbow`

que representa el landmark del codo izquierdo.

La función recibe estos dos puntos y calcula el vector entre ellos.

Conceptualmente:

`left_shoulder`

|
|
↓

`left_elbow`

El resultado se guarda en:

`left_arm`

Por tanto:

`left_arm = vector(Hombro izquierdo → Codo izquierdo)`

Este vector representa la **orientación del brazo superior izquierdo**.

Vector del antebrazo izquierdo

```
left_forearm = calculate_vector(  
    pose.left_elbow,  
    pose.left_wrist  
)
```

```
pose.left_elbow,  
pose.left_wrist  
)
```

A continuación calculamos el vector correspondiente al antebrazo izquierdo.

En este caso utilizamos:

```
pose.left_elbow  
como punto inicial,  
y:  
pose.left_wrist  
como punto final.
```

Por tanto:

```
left_forearm =  
vector(Codo izquierdo → Muñeca izquierda)
```

Conceptualmente:

Hombro

↓

Codo

↓

Muñeca

Hombro → Codo = left_arm

Codo → Muñeca = left_forearm

De esta manera obtenemos por separado los dos segmentos principales del brazo izquierdo.

Brazo derecho

```
# -----  
# Brazo derecho  
# -----
```

A continuación realizamos exactamente el mismo proceso para el brazo derecho.

La estructura del cálculo es idéntica, pero utilizando los landmarks correspondientes al lado derecho.

Vector del brazo derecho

```
right_arm = calculate_vector(  
    pose.right_shoulder,  
    pose.right_elbow  
)
```

Aquí calculamos el vector que va desde el hombro derecho hasta el codo derecho.

Por tanto:

```
right_arm =  
vector(Hombro derecho → Codo derecho)
```

Este vector representa la **orientación del brazo superior derecho**.

Vector del antebrazo derecho

```
right_forearm = calculate_vector(  
    pose.right_elbow,  
    pose.right_wrist  
)
```

Finalmente calculamos el vector correspondiente al antebrazo derecho.

El punto inicial es:

```
pose.right_elbow
```

y el punto final:

```
pose.right_wrist
```

Por tanto:

```
right_forearm =  
vector(Codo derecho → Muñeca derecha)
```

La estructura completa del brazo derecho queda:

Hombro derecho



Codo derecho



Muñeca derecha

Hombro → Codo = right_arm

Codo → Muñeca = right_forearm

Devolver vectores relativos

```
# -----
```

```
# Devolver vectores relativos
```

```
# -----
```

Una vez calculados los cuatro vectores, la función debe devolverlos para que otras partes del proyecto puedan utilizarlos.

Para ello se utiliza:

```
return {
```

La función devuelve un **diccionario de Python**.

Este diccionario permite organizar los resultados de una manera clara, separando la información del brazo izquierdo y del brazo derecho.

Estructura del resultado

```
return {
```

```
    "left_arm": {
```

```
        "upper_arm": left_arm,
```

```
        "forearm": left_forearm
```

```
    },
```

```
    "right_arm": {
```

```
        "upper_arm": right_arm,
```

```
        "forearm": right_forearm
```

```
    }
```

```
}
```

La estructura devuelta tiene dos grandes bloques:

left_arm

right_arm

Cada uno contiene a su vez dos vectores:

upper_arm

forearm

Por tanto, podemos representarlo como:

Resultado

```

|
|  └─ left_arm
|
|  └─ upper_arm → vector hombro → codo
|
|  └─ forearm  → vector codo → muñeca
|
└─ right_arm
    └─ upper_arm → vector hombro → codo
    └─ forearm  → vector codo → muñeca

```

¿Por qué se utiliza upper_arm en lugar de left_arm?

Durante el cálculo utilizamos:

left_arm

para almacenar temporalmente el vector del brazo izquierdo.

Sin embargo, al devolverlo utilizamos:

"upper_arm": left_arm

Esto permite que la estructura final sea más clara.

upper_arm indica específicamente que estamos hablando del **brazo superior**, es decir, del segmento:

Hombro → Codo

Mientras que:

"forearm"

representa:

Codo → Muñeca

Esto evita confundir el concepto de "brazo completo" con cada uno de sus segmentos.

Ejemplo conceptual

Supongamos que tenemos:

Hombro izquierdo = (100, 100)

Codo izquierdo = (150, 150)

Muñeca izquierda = (180, 120)

La función calcularía:

Hombro → Codo

↓

(50, 50)

Codo → Muñeca

↓

(30, -30)

Estos vectores describen la orientación de los diferentes segmentos del brazo.

La función devolvería una estructura conceptualmente similar a:

```
{  
  "left_arm": {  
    "upper_arm": (50, 50),  
    "forearm": (30, -30)  
  },  
  
  "right_arm": {  
    "upper_arm": ...,  
    "forearm": ...  
  }  
}
```

Los valores concretos dependerán de las coordenadas detectadas en cada instante.

¿Por qué calculamos posiciones relativas?

Este paso es importante porque las coordenadas absolutas de la imagen dependen de dónde se encuentre la persona dentro del frame.

Por ejemplo, una persona puede mover todo su cuerpo hacia la derecha:

Antes:

Hombro → (100,100)

Codo → (150,150)

Después:

Hombro → (300,100)

Codo → (350,150)

Las coordenadas han cambiado, pero la orientación del brazo sigue siendo la misma.

Los vectores relativos permiten representar esta relación:

Hombro → Codo

en lugar de depender exclusivamente de la posición absoluta dentro de la imagen.

Esto resulta especialmente interesante para el objetivo del proyecto, ya que el sistema busca interpretar **movimientos del operador** y posteriormente utilizar esa información para controlar un sistema robótico.

Relación con la capa matemática

Este archivo actúa como una capa intermedia entre la representación de la pose y los cálculos matemáticos.

La arquitectura puede entenderse así:

Landmark



HumanPose



relative_pose.py



calculate_vector()



Vectores de los segmentos



Análisis de movimiento

Landmark almacena los puntos.

HumanPose organiza esos puntos.

relative_pose.py selecciona los puntos necesarios para representar los segmentos de los brazos.

calculate_vector() realiza el cálculo matemático.

El resultado son los vectores que representan la orientación de:

- brazo superior izquierdo;
- antebrazo izquierdo;
- brazo superior derecho;

- antebrazo derecho.

Importancia dentro del proyecto

`calculate_relative_arm_pose()` constituye un paso importante para pasar de **puntos detectados en una imagen** a una representación matemática del movimiento humano.

El sistema deja de trabajar únicamente con:

Hombro

Codo

Muñeca

como puntos independientes y comienza a trabajar con:

Hombro → Codo

Codo → Muñeca

como segmentos orientados.

Esto proporciona una representación más útil para las siguientes etapas del proyecto, especialmente para el análisis de la postura y el movimiento de los brazos.

En resumen:

Detección de landmarks



Hombro

Codo

Muñeca



Cálculo de vectores



Brazo superior

Antebrazo



Representación relativa



Análisis matemático

Por tanto, la función `calculate_relative_arm_pose()` no realiza la detección de la persona. Su función es tomar los landmarks que ya han sido obtenidos y convertir sus posiciones en **vectores relativos que describen la orientación de los segmentos de ambos brazos.**

Limits.py

```
# =====  
  
# MOVEMENT LIMITS  
  
# =====  
  
# Valores provisionales.  
  
# Se definirán correctamente cuando se conozca  
# el robot y la configuración final del sistema.  
  
LEFT_ELBOW_MIN = 0  
LEFT_ELBOW_MAX = 360  
  
RIGHT_ELBOW_MIN = 0  
RIGHT_ELBOW_MAX = 360
```

Limits.py Explicación

Límites de movimiento

```
# =====  
  
# MOVEMENT LIMITS  
  
# =====
```

Este bloque identifica la finalidad del archivo: definir los **límites de movimiento** que posteriormente podrán utilizarse para determinar si una posición o movimiento del operador se encuentra dentro de los valores permitidos.

Los límites forman parte de la capa encargada de establecer **qué movimientos son válidos y cuáles deberían considerarse fuera del rango permitido**.

En el estado actual del proyecto, estos valores todavía no están definidos de acuerdo con un robot concreto.

Valores provisionales

```
# Valores provisionales.  
  
# Se definirán correctamente cuando se conozca  
# el robot y la configuración final del sistema.
```

Estos comentarios son especialmente importantes porque indican que los valores definidos en este archivo son **provisionales**.

Actualmente todavía no se ha establecido de forma definitiva:

- el modelo de robot que se utilizará;
- la configuración final de sus articulaciones;
- los rangos de movimiento de cada articulación;
- la correspondencia exacta entre el movimiento humano y el movimiento del robot.

Por este motivo, no tendría sentido establecer todavía unos límites físicos definitivos.

La intención es que esta información pueda modificarse posteriormente cuando se conozca la configuración final del sistema.

Límite del codo izquierdo

LEFT_ELBOW_MIN = 0

LEFT_ELBOW_MAX = 360

Estas dos variables definen actualmente el rango provisional asociado al **codo izquierdo**.

LEFT_ELBOW_MIN representa el límite mínimo:

LEFT_ELBOW_MIN = 0

Mientras que:

LEFT_ELBOW_MAX = 360

representa el límite máximo.

Por tanto, provisionalmente:

Codo izquierdo

|

|— Mínimo → 0°

└— Máximo → 360°

Estos valores permiten establecer una estructura inicial para que posteriormente el sistema pueda comprobar si un ángulo calculado se encuentra dentro de los límites establecidos.

Sin embargo, **no deben interpretarse todavía como los límites físicos reales del codo humano ni de una articulación robótica**.

El rango 0–360 se utiliza en esta fase como un valor provisional mientras se define la configuración final.

Límite del codo derecho

RIGHT_ELBOW_MIN = 0

RIGHT_ELBOW_MAX = 360

De forma equivalente, se definen los límites provisionales del **codo derecho**.

Tenemos:

`RIGHT_ELLOW_MIN = 0`

como límite mínimo,

y:

`RIGHT_ELLOW_MAX = 360`

como límite máximo.

La estructura es:

Codo derecho

|

|— Mínimo → 0°

└─ Máximo → 360°

Al igual que ocurre con el brazo izquierdo, estos valores son provisionales y podrán modificarse posteriormente.

Separación entre brazo izquierdo y derecho

Un detalle importante es que los límites se almacenan de forma independiente para cada brazo.

Tenemos:

`LEFT_ELLOW_MIN`

`LEFT_ELLOW_MAX`

para el lado izquierdo,

y:

`RIGHT_ELLOW_MIN`

`RIGHT_ELLOW_MAX`

para el lado derecho.

Esto permite que en el futuro cada brazo pueda tener **restricciones diferentes** si la configuración del sistema lo requiere.

Por ejemplo, una configuración futura podría establecer distintos rangos para cada lado:

Brazo izquierdo

↓

Límite mínimo

Límite máximo

Brazo derecho



Límite mínimo

Límite máximo

De esta forma, el sistema no queda limitado a utilizar un único conjunto de valores para ambos brazos.

Relación con PoseMath

Este archivo está directamente relacionado con la información calculada anteriormente mediante PoseMath.

En PoseMath obtenemos información matemática sobre la pose, como los **ángulos de las articulaciones**.

El archivo Limits.py proporciona posteriormente los valores que pueden utilizarse para determinar si esos resultados se encuentran dentro de un rango permitido.

Conceptualmente:

Landmark



HumanPose



Relative Pose



PoseMath



Ángulos y vectores



Limits



Comprobación de rangos

Por tanto, Limits.py no calcula el ángulo del codo.

Su función es **proporcionar los límites contra los que posteriormente se podrá comparar ese ángulo**.

Relación con Robot Mapping

Los límites también serán importantes en las siguientes etapas del proyecto, especialmente cuando se establezca la correspondencia entre el movimiento humano y el robot.

La idea general será:

Movimiento humano



Detección de landmarks



Cálculo matemático



Ángulo / posición



Comprobación de límites



Robot Mapping



Movimiento del robot

Esto permite evitar que una determinada orden de movimiento se traduzca directamente al robot sin comprobar previamente si se encuentra dentro de los rangos establecidos.

Estado actual de los límites

En el estado actual del proyecto, Limits.py debe entenderse como una **estructura preparada para incorporar las restricciones definitivas del sistema**, pero todavía no como una implementación completa de los límites físicos del robot.

Los valores actuales:

0

y:

360

son valores provisionales.

Cuando se conozca el robot y su configuración final, deberán sustituirse por los rangos correspondientes.

Por tanto, este archivo funciona actualmente como una **base de configuración** sobre la que posteriormente se podrán implementar las restricciones reales.

Importancia dentro del proyecto

Aunque el archivo es pequeño, introduce un concepto importante: el sistema no debe limitarse a detectar y calcular movimientos, sino que también debe poder **determinar si esos movimientos son válidos dentro de las restricciones establecidas**.

La separación en un archivo independiente permite modificar los límites sin tener que cambiar directamente la lógica matemática o la lógica de detección.

De esta manera:

PoseMath.py

→ calcula

Limits.py

→ define los rangos permitidos

Robot Mapping.py

→ establece la correspondencia con el robot

Robot Motion.py

→ ejecuta el movimiento

Esta separación facilita que el sistema pueda adaptarse posteriormente a diferentes configuraciones de robot.

Por tanto, Limits.py establece la estructura inicial para controlar los rangos de movimiento del sistema. En esta fase los límites son provisionales y deberán definirse correctamente cuando se conozcan el robot y la configuración final de Kinema Nexus.

Robot_mapping.py

```
import json

from pathlib import Path

# =====

# CONFIGURACIÓN

# =====

CALIBRATION_FILE = (
    Path(__file__).resolve().parent.parent
    / "data"
    / "calibration.json"
)

# =====

# MEDIDAS PROVISIONALES DEL ROBOT

# =====

# Estos valores son únicamente para pruebas.
# Se sustituirán cuando se defina el robot definitivo.

ROBOT_LEFT_ARM_LENGTH = 300.0
ROBOT_RIGHT_ARM_LENGTH = 300.0

ROBOT_LEFT_FOREARM_LENGTH = 280.0
ROBOT_RIGHT_FOREARM_LENGTH = 280.0

# =====

# CARGAR CALIBRACIÓN

# =====
```

```

def load_calibration():

    if not CALIBRATION_FILE.exists():

        raise FileNotFoundError(

            f"No se encontró el archivo de calibración: "

            f"{CALIBRATION_FILE}"

        )

    with open(

        CALIBRATION_FILE,

        "r",

        encoding="utf-8"

    ) as file:

        return json.load(file)

# =====

# CALCULAR ESCALA

# =====

def calculate_scale(

    robot_length,

    human_length

):

    if human_length <= 0:

        raise ValueError(

            "La longitud humana debe ser mayor que cero."

        )

```

```
return robot_length / human_length
```

```
# =====
```

```
# CREAR MAPEO HUMANO - ROBOT
```

```
# =====
```

```
def create_robot_mapping(calibration):
```

```
    # -----
```

```
    # Medidas del operador
```

```
    # -----
```

```
    human_left_arm = calibration["left_arm_length"]
```

```
    human_right_arm = calibration["right_arm_length"]
```

```
    human_left_forearm = calibration["left_forearm_length"]
```

```
    human_right_forearm = calibration["right_forearm_length"]
```

```
    # -----
```

```
    # Calcular relaciones
```

```
    # -----
```

```
    left_arm_scale = calculate_scale(
```

```
        ROBOT_LEFT_ARM_LENGTH,
```

```
        human_left_arm
```

```
)
```

```
    right_arm_scale = calculate_scale(
```

```
        ROBOT_RIGHT_ARM_LENGTH,
```

```
        human_right_arm
```

```
)
```

```
left_forearm_scale = calculate_scale(  
    ROBOT_LEFT_FOREARM_LENGTH,  
    human_left_forearm  
)
```

```
right_forearm_scale = calculate_scale(  
    ROBOT_RIGHT_FOREARM_LENGTH,  
    human_right_forearm  
)
```

```
# -----  
# Devolver configuración de mapeo  
# -----
```

```
return {  
    "left_arm_scale": left_arm_scale,  
    "right_arm_scale": right_arm_scale,  
    "left_forearm_scale": left_forearm_scale,  
    "right_forearm_scale": right_forearm_scale  
}
```

```
# =====  
# CARGAR Y CREAR MAPEO  
# =====
```

```
def load_robot_mapping():
```

```
    calibration = load_calibration()
```



```
mapping = create_robot_mapping(  
    calibration  
)
```

```
return mapping
```

robot_mapping.py explicación

Importación de librerías

```
import json
```

```
from pathlib import Path
```

En este archivo se importan dos librerías de Python.

json permite trabajar con archivos en formato JSON. En este caso se utilizará para **leer el archivo de calibración** generado previamente.

Path, de la librería pathlib, permite trabajar con rutas de archivos de una manera más estructurada y portable.

Estas dos librerías se utilizan principalmente para localizar y cargar la información de calibración del operador.

Configuración

```
CALIBRATION_FILE = (  
    Path(__file__).resolve().parent.parent  
    / "data"  
    / "calibration.json"  
)
```

Esta variable almacena la ruta del archivo de calibración que utilizará el sistema.

La expresión:

```
Path(__file__).resolve()
```

obtiene la ruta absoluta del archivo Python que se está ejecutando.

A continuación:

```
.parent.parent
```

permite subir dos niveles dentro de la estructura de directorios del proyecto.

Finalmente se añaden:

```
/data/calibration.json
```

Por tanto, el programa busca el archivo:

data/

└─ calibration.json

Este archivo contiene las medidas obtenidas durante la fase de calibración del operador.

La relación entre ambos archivos es importante:

Calibration.py



calibration.json



Robot Mapping.py

Calibration.py se encarga de obtener y guardar las medidas del operador, mientras que Robot Mapping.py recupera esas medidas para utilizarlas en el proceso de correspondencia con el robot.

Medidas provisionales del robot

```
# =====
```

```
# MEDIDAS PROVISIONALES DEL ROBOT
```

```
# =====
```

Este bloque contiene las dimensiones que actualmente se utilizan como referencia para el robot.

```
# Estos valores son únicamente para pruebas.
```

```
# Se sustituirán cuando se defina el robot definitivo.
```

Al igual que ocurría en Limits.py, estos valores son **provisionales**.

Actualmente el proyecto todavía no está vinculado a una configuración física definitiva del robot. Por ello, se utilizan unas dimensiones ficticias para poder desarrollar y probar la lógica de mapeo.

Longitud del brazo del robot

```
ROBOT_LEFT_ARM_LENGTH = 300.0
```

```
ROBOT_RIGHT_ARM_LENGTH = 300.0
```

Estas variables representan provisionalmente la longitud del brazo superior del robot.

Se mantiene una variable independiente para cada lado:

```
ROBOT_LEFT_ARM_LENGTH
```

→ brazo superior izquierdo

```
ROBOT_RIGHT_ARM_LENGTH
```

→ brazo superior derecho

En este caso ambos tienen un valor de:

300.0

La unidad dependerá de la configuración que se establezca para el robot, pero conceptualmente representa una **longitud física del segmento robótico**.

Longitud del antebrazo del robot

ROBOT_LEFT_FOREARM_LENGTH = 280.0

ROBOT_RIGHT_FOREARM_LENGTH = 280.0

Estas variables representan la longitud provisional de los antebrazos del robot.

Por tanto:

ROBOT_LEFT_FOREARM_LENGTH

→ antebrazo izquierdo

ROBOT_RIGHT_FOREARM_LENGTH

→ antebrazo derecho

Actualmente ambos tienen un valor de:

280.0

De esta forma, el sistema dispone de una referencia de longitud para cada uno de los cuatro segmentos:

Robot

|

├— Brazo izquierdo → 300

├— Antebrazo izquierdo → 280

├— Brazo derecho → 300

└— Antebrazo derecho → 280

Cuando se conozca el robot definitivo, estos valores deberán sustituirse por las dimensiones reales.

Cargar calibración

def load_calibration():

Esta función se encarga de **cargar los datos de calibración del operador**.

La función no recibe ningún parámetro porque la ubicación del archivo ya está definida anteriormente mediante:

CALIBRATION_FILE

Comprobar que existe el archivo

```
if not CALIBRATION_FILE.exists():
```

Antes de intentar abrir el archivo, el programa comprueba si realmente existe.

Esto evita intentar leer un archivo que no está disponible.

Si el archivo no existe, se ejecuta:

```
raise FileNotFoundError(  
    f"No se encontró el archivo de calibración: "  
    f"{CALIBRATION_FILE}"  
)
```

Aquí se genera una excepción `FileNotFoundError`.

Esto permite informar claramente de que el sistema no ha podido encontrar el archivo de calibración.

El mensaje incluye además la ruta que se estaba intentando utilizar.

Esto es útil durante la ejecución y especialmente durante la depuración del proyecto.

Abrir el archivo de calibración

```
with open(  
    CALIBRATION_FILE,  
    "r",  
    encoding="utf-8"  
) as file:
```

Si el archivo existe, se abre en modo lectura.

El parámetro:

"r"

significa *read*, es decir, lectura.

Además:

encoding="utf-8"

indica la codificación utilizada para interpretar el contenido del archivo.

El uso de:

```
with open(...)
```

hace que el archivo se gestione correctamente y se cierre automáticamente al terminar de utilizarlo.

Leer los datos JSON

```
return json.load(file)
```

Esta línea utiliza la librería json para convertir el contenido del archivo JSON en una estructura de datos de Python.

Por ejemplo, el archivo podría contener información conceptualmente similar a:

```
{  
    "left_arm_length": ...,  
    "right_arm_length": ...,  
    "left_forearm_length": ...,  
    "right_forearm_length": ...  
}
```

Después de utilizar json.load(), estos datos pueden accederse desde Python mediante sus respectivas claves.

Por tanto, load_calibration() devuelve las medidas del operador almacenadas durante la calibración.

Calcular escala

```
def calculate_scale(  
    robot_length,  
    human_length  
):
```

Esta función calcula la **relación de escala entre una dimensión del robot y la dimensión correspondiente del operador**.

Recibe dos valores:

robot_length

→ longitud del segmento correspondiente en el robot.

human_length

→ longitud del segmento correspondiente del operador.

La idea matemática es:

escala = longitud del robot / longitud humana

Esta relación permite determinar cuánto debe escalarse una medida obtenida del operador para adaptarla al tamaño del robot.

Comprobar la longitud humana

```
if human_length <= 0:
```

Antes de realizar la división, se comprueba que la longitud del operador sea válida.

Una longitud igual o menor que cero no tendría sentido en este contexto.

Si se detecta un valor incorrecto:

```
raise ValueError(  
    "La longitud humana debe ser mayor que cero."  
)
```

se genera una excepción ValueError.

Esto evita realizar una división incorrecta y permite detectar posibles errores en los datos de calibración.

Calcular la escala

```
return robot_length / human_length
```

Finalmente se realiza la operación:

```
escala = robot_length / human_length
```

Por ejemplo, si:

Longitud humana = 400

Longitud robot = 300

la escala sería:

$300 / 400 = 0.75$

Esto significa que la dimensión correspondiente del robot es un 75 % de la dimensión humana utilizada como referencia.

El objetivo no es modificar físicamente al operador, sino obtener un **factor matemático de correspondencia** que posteriormente pueda utilizarse para transformar el movimiento.

Crear mapeo humano-robot

```
def create_robot_mapping(calibration):
```

Esta función constituye el núcleo del archivo.

Su objetivo es utilizar las medidas obtenidas durante la calibración para crear una configuración que permita relacionar las dimensiones del operador con las dimensiones provisionales del robot.

Recibe:

calibration

que contiene los datos cargados desde calibration.json.

Obtener las medidas del operador

```
human_left_arm = calibration["left_arm_length"]
```

```
human_right_arm = calibration["right_arm_length"]
```

```
human_left_forearm = calibration["left_forearm_length"]
```

```
human_right_forearm = calibration["right_forearm_length"]
```

Aquí extraemos del diccionario de calibración las cuatro medidas correspondientes al operador.

Tenemos:

```
human_left_arm
```

→ longitud del brazo izquierdo

```
human_right_arm
```

→ longitud del brazo derecho

```
human_left_forearm
```

→ longitud del antebrazo izquierdo

```
human_right_forearm
```

→ longitud del antebrazo derecho

Estas medidas proceden de la fase de calibración.

Por tanto, aquí se produce una conexión directa con el trabajo realizado anteriormente en Calibration.py.

Calcular escala del brazo izquierdo

```
left_arm_scale = calculate_scale(  
    ROBOT_LEFT_ARM_LENGTH,  
    human_left_arm  
)
```

Se calcula la relación entre:

brazo izquierdo del robot

y:

brazo izquierdo del operador

La función utiliza:

```
ROBOT_LEFT_ARM_LENGTH
```

como longitud objetivo,

y:

human_left_arm

como longitud de referencia humana.

El resultado se almacena en:

left_arm_scale

Calcular escala del brazo derecho

```
right_arm_scale = calculate_scale(  
    ROBOT_RIGHT_ARM_LENGTH,  
    human_right_arm  
)
```

Se realiza el mismo cálculo para el brazo derecho.

Obtenemos:

right_arm_scale

que representa la relación entre la longitud del brazo derecho del robot y la del operador.

Calcular escala del antebrazo izquierdo

```
left_forearm_scale = calculate_scale(  
    ROBOT_LEFT_FOREARM_LENGTH,  
    human_left_forearm  
)
```

Aquí se calcula la relación entre el antebrazo izquierdo del robot y el del operador.

El resultado se almacena en:

left_forearm_scale

Calcular escala del antebrazo derecho

```
right_forearm_scale = calculate_scale(  
    ROBOT_RIGHT_FOREARM_LENGTH,  
    human_right_forearm  
)
```

Finalmente se calcula la escala correspondiente al antebrazo derecho.

El resultado queda almacenado en:

right_forearm_scale

De esta manera obtenemos cuatro factores de escala independientes:

left_arm_scale

right_arm_scale

left_forearm_scale

right_forearm_scale

Devolver configuración de mapeo

```
return {  
    "left_arm_scale": left_arm_scale,  
    "right_arm_scale": right_arm_scale,  
    "left_forearm_scale": left_forearm_scale,  
    "right_forearm_scale": right_forearm_scale  
}
```

Una vez calculados los cuatro factores, la función los agrupa en un diccionario.

La estructura resultante es:

Robot Mapping

|

├─ left_arm_scale

├─ right_arm_scale

├─ left_forearm_scale

└─ right_forearm_scale

Esta estructura permite que otras partes del proyecto puedan acceder fácilmente a los factores de escala.

Cargar y crear el mapeo

```
def load_robot_mapping():
```

Esta función proporciona una forma sencilla de obtener directamente el mapeo completo.

Internamente realiza dos pasos:

1. cargar la calibración;
2. crear el mapeo a partir de ella.

Cargar la calibración

```
calibration = load_calibration()
```

Primero se llama a load_calibration().

Esto recupera las medidas del operador desde:

data/calibration.json

El resultado se almacena en:

calibration

Crear el mapeo

```
mapping = create_robot_mapping(  
    calibration  
)
```

A continuación, las medidas obtenidas se pasan a `create_robot_mapping()`.

Esta función calcula los cuatro factores de escala necesarios para relacionar las dimensiones humanas con las del robot.

Devolver el mapeo

```
return mapping
```

Finalmente se devuelve el diccionario generado.

De esta forma, desde otras partes del proyecto se puede obtener todo el mapeo mediante una única función:

```
load_robot_mapping()
```

↓

```
load_calibration()
```

↓

```
create_robot_mapping()
```

↓

Factores de escala

Flujo completo del archivo

El funcionamiento completo de `Robot Mapping.py` puede resumirse de la siguiente manera:

Calibration.py

↓

calibration.json

↓

```
load_calibration()
```

↓

Medidas del operador



create_robot_mapping()



Comparación con medidas del robot



calculate_scale()



Factores de escala



Robot Mapping

Se generan cuatro relaciones:

Operador $\leftrightarrow$ Robot

Brazo izquierdo

Brazo derecho

Antebrazo izquierdo

Antebrazo derecho

Relación con Relative Pose

Este archivo utiliza una información diferente a la obtenida en Relative Pose.

Relative Pose trabaja principalmente con la **orientación relativa de los segmentos**, mediante vectores:

Hombro $\rightarrow$ Codo

Codo $\rightarrow$ Muñeca

Mientras que Robot Mapping introduce la **relación de escala de las dimensiones físicas**.

Podemos verlo como:

Relative Pose



Orientación de los segmentos

Robot Mapping



Escala de los segmentos

Ambas informaciones serán necesarias para conseguir una correspondencia más completa entre el movimiento humano y el movimiento robótico.

Relación con Calibration.py

La calibración es la fuente de las medidas humanas utilizadas en este archivo.

El flujo es:

Calibration.py



Medidas del operador



calibration.json



Robot Mapping.py

Esto permite que las medidas no estén escritas directamente dentro del código de mapeo, sino que se obtengan desde el archivo de configuración generado durante la calibración.

Relación con Robot Motion

El resultado de este archivo está pensado para ser utilizado posteriormente por la capa de movimiento del robot.

Conceptualmente:

Human Pose



Relative Pose



Información del movimiento



Robot Mapping



Adaptación a dimensiones del robot



Robot Motion



Movimiento del robot

Robot Mapping.py se encarga de establecer **cómo se relaciona el movimiento humano con las dimensiones del robot**, mientras que Robot Motion.py será la parte encargada de utilizar esa información para generar el movimiento correspondiente.

Estado actual del mapeo

Es importante destacar que el mapeo implementado actualmente todavía constituye una **primera aproximación basada en escalas de longitud**.

Las dimensiones del robot:

300.0

280.0

son valores provisionales y no corresponden todavía necesariamente a un robot físico concreto.

Por tanto, el código actual permite desarrollar y probar la lógica de correspondencia, pero la configuración definitiva deberá realizarse cuando se conozcan:

- el robot utilizado;
- sus dimensiones reales;
- la configuración de sus articulaciones;
- sus límites de movimiento;
- y la correspondencia final entre los movimientos humanos y las articulaciones robóticas.

Por tanto, Robot Mapping.py constituye la capa encargada de transformar las medidas obtenidas durante la calibración del operador en factores de escala que permiten establecer una primera correspondencia dimensional entre el cuerpo humano y el robot.

Robot_motion.py

```
import json

from pathlib import Path

import math


from pose_math import calculate_angle


# =====
# CONFIGURACIÓN
# =====

OUTPUT_FILE = (
    Path(__file__).resolve().parent.parent
    / "data"
    / "robot_motion.json"
)


# =====
# LIMITES PROVISIONALES DEL ROBOT
# =====

J1_MIN = -180.0
J1_MAX = 180.0

J2_MIN = -180.0
J2_MAX = 180.0

J3_MIN = -180.0
J3_MAX = 180.0


# =====
```

```
# LIMITAR VALOR
```

```
# =====
```

```
def clamp(value, minimum, maximum):
```

```
    return max(
        minimum,
        min(value, maximum)
    )
```

```
# =====
```

```
# CALCULAR MOVIMIENTO DEL ROBOT
```

```
# =====
```

```
def calculate_robot_motion(relative_pose):
```

```
    # -----
```

```
    # Obtener vectores del brazo derecho
```

```
    # -----
```

```
    upper_arm = relative_pose["right_arm"]["upper_arm"]
```

```
    forearm = relative_pose["right_arm"]["forearm"]
```

```
    # -----
```

```
    # Ángulo del brazo respecto al eje X
```

```
    # -----
```

```
    shoulder_angle = math.degrees(
```

```
        math.atan2(
            upper_arm[1],
            upper_arm[0]
```

```

    )
)

# -----
# Ángulo entre brazo y antebrazo
# -----

elbow_angle = calculate_angle(
    upper_arm,
    forearm
)

# -----
# Conversión provisional a articulaciones del UR
# -----

j1 = shoulder_angle

j2 = -shoulder_angle

j3 = 180.0 - elbow_angle

# -----
# Aplicar límites
# -----

j1 = clamp(
    j1,
    J1_MIN,
    J1_MAX
)

```



```
j2 = clamp(  
    j2,  
    J2_MIN,  
    J2_MAX  
)
```

```
j3 = clamp(  
    j3,  
    J3_MIN,  
    J3_MAX  
)
```

```
# -----  
# Crear configuración  
# -----
```

```
return {  
    "joints": [  
        j1,  
        j2,  
        j3,  
        0.0,  
        0.0,  
        0.0  
    ]  
}
```

```
# =====  
# GUARDAR JSON  
# =====
```

```
def save_robot_motion(robot_motion):
```

```
    with open(
```

```
        OUTPUT_FILE,
```

```
        "w",
```

```
        encoding="utf-8"
```

```
    ) as file:
```

```
        json.dump(
```

```
            robot_motion,
```

```
            file,
```

```
            indent=4
```

```
        )
```

```
# =====
```

```
# FUNCIÓN PRINCIPAL
```

```
# =====
```

```
def update_robot_motion(relative_pose):
```

```
    robot_motion = calculate_robot_motion(
```

```
        relative_pose
```

```
    )
```

```
    save_robot_motion(
```

```
        robot_motion
```

```
    )
```

```
    return robot_motion
```

robot_motion.py explicación

Importación de librerías

```
import json
```

```
from pathlib import Path
```

```
import math
```

```
from pose_math import calculate_angle
```

Este archivo utiliza varias herramientas necesarias para calcular y almacenar el movimiento del robot.

json

```
import json
```

La librería json permite trabajar con archivos en formato JSON.

En este caso se utilizará para **guardar la configuración de movimiento calculada para el robot**.

El resultado será almacenado posteriormente en:

```
data/robot_motion.json
```

Path

```
from pathlib import Path
```

Path permite trabajar con rutas de archivos de una forma estructurada.

Se utilizará para construir la ubicación del archivo donde se almacenará la configuración del movimiento.

math

```
import math
```

La librería math proporciona funciones matemáticas necesarias para realizar cálculos relacionados con los ángulos.

En este archivo se utilizan principalmente:

```
math.atan2()
```

para obtener la orientación de un vector,

y:

```
math.degrees()
```

para convertir el resultado de radianes a grados.

calculate_angle

```
from pose_math import calculate_angle
```

Aquí importamos la función `calculate_angle` que ya había sido definida en `pose_math.py`.

Esta función permite calcular el ángulo existente entre dos vectores.

Por tanto, este archivo reutiliza la capa matemática existente en lugar de volver a implementar el cálculo.

La relación es:

`pose_math.py`



`calculate_angle()`



`Robot Motion.py`

Configuración

```
OUTPUT_FILE = (  
    Path(__file__).resolve().parent.parent  
    / "data"  
    / "robot_motion.json"  
)
```

Esta variable define dónde se guardará el resultado del cálculo del movimiento del robot.

Al igual que ocurría con `calibration.json`, se utiliza `Path` para construir la ruta a partir de la ubicación del propio archivo.

El resultado apunta a:

`data/`

└─ `robot_motion.json`

Este archivo contendrá la configuración de las articulaciones calculada por el sistema.

Por tanto, este archivo JSON representa una de las salidas de esta etapa del procesamiento.

Límites provisionales del robot

`J1_MIN = -180.0`

`J1_MAX = 180.0`

`J2_MIN = -180.0`

`J2_MAX = 180.0`

`J3_MIN = -180.0`

J3_MAX = 180.0

Estas variables establecen los límites provisionales de las tres primeras articulaciones utilizadas en esta implementación.

Tenemos:

J1 → -180° a 180°

J2 → -180° a 180°

J3 → -180° a 180°

Estos valores son provisionales y se utilizan para evitar que el sistema genere valores de articulación fuera del rango establecido.

Al igual que en Limits.py, estos límites deberán adaptarse cuando se conozca la configuración definitiva del robot.

Es importante destacar que estos valores **no representan todavía necesariamente los límites físicos reales del robot definitivo**.

Función clamp

```
def clamp(value, minimum, maximum):
```

La función clamp se utiliza para garantizar que un valor permanezca dentro de un rango determinado.

Recibe tres parámetros:

value

minimum

maximum

donde:

- value → valor que queremos limitar.
- minimum → límite inferior permitido.
- maximum → límite superior permitido.

Aplicación del límite

```
return max(  
    minimum,  
    min(value, maximum)  
)
```

Aquí se utiliza una combinación de min() y max().

La operación puede entenderse en dos pasos.

Primero:

`min(value, maximum)`

impide que el valor sea superior al máximo.

Después:

`max(minimum, ...)`

impide que el resultado sea inferior al mínimo.

Por tanto, el resultado siempre queda dentro del intervalo:

$\text{minimum} \leq \text{resultado} \leq \text{maximum}$

Por ejemplo, si tenemos:

`minimum = -180`

`maximum = 180`

y recibimos:

250

el resultado será:

180

Si recibimos:

-220

el resultado será:

-180

Y si recibimos:

90

el resultado seguirá siendo:

90

Esta función será utilizada posteriormente para proteger las articulaciones del robot frente a valores fuera de los límites definidos.

Calcular movimiento del robot

`def calculate_robot_motion(relative_pose):`

Esta es la función principal encargada de transformar la información de la pose relativa humana en una configuración de movimiento del robot.

Recibe:

`relative_pose`

que contiene los vectores calculados anteriormente mediante Relative Pose.

La cadena de procesamiento comienza a tomar forma:

HumanPose



Relative Pose



Vectores de los brazos



Robot Motion



Ángulos de articulaciones

Obtener vectores del brazo derecho

```
upper_arm = relative_pose["right_arm"]["upper_arm"]
```

```
forearm = relative_pose["right_arm"]["forearm"]
```

En esta implementación se utilizan los vectores correspondientes al **brazo derecho**.

Se obtiene:

```
relative_pose["right_arm"]["upper_arm"]
```

que representa el vector:

Hombro derecho → Codo derecho

y:

```
relative_pose["right_arm"]["forearm"]
```

que representa:

Codo derecho → Muñeca derecha

Por tanto:

```
upper_arm
```



Brazo superior

```
forearm
```



Antebrazo

Estos vectores proceden directamente de Relative Pose.

Ángulo del brazo respecto al eje X

```
shoulder_angle = math.degrees(  
    math.atan2(  
        upper_arm[1],  
        upper_arm[0]  
    )  
)
```

Aquí se calcula la orientación del brazo superior respecto al eje X de la imagen.

El vector:

`upper_arm = (x, y)`

contiene dos componentes:

`upper_arm[0]` → componente X

`upper_arm[1]` → componente Y

La función:

`math.atan2(y, x)`

calcula el ángulo del vector teniendo en cuenta el cuadrante en el que se encuentra.

El resultado de `atan2()` se proporciona en **radianes**.

Por este motivo se utiliza:

`math.degrees()`

para convertir el resultado a grados.

El resultado final se almacena en:

`shoulder_angle`

Por tanto:

Vector brazo superior

↓

`atan2()`

↓

Ángulo en radianes

↓

`degrees()`

↓

`shoulder_angle`

Este ángulo representa la orientación del brazo superior.

Ángulo entre brazo y antebrazo

```
elbow_angle = calculate_angle(  
    upper_arm,  
    forearm  
)
```

A continuación calculamos el ángulo existente entre el brazo superior y el antebrazo.

Para ello se utiliza la función:

```
calculate_angle()
```

que ya había sido implementada en pose_math.py.

Se le proporcionan:

```
upper_arm
```

```
forearm
```

es decir:

Hombro → Codo

Codo → Muñeca

El resultado:

```
elbow_angle
```

representa el ángulo calculado entre ambos segmentos.

Esto permite obtener una representación matemática de la articulación del codo.

La relación entre los dos vectores es:

Hombro



Codo



Muñeca

upper_arm



Codo



forearm

Conversión provisional a articulaciones del robot

$j1 = \text{shoulder_angle}$

$j2 = -\text{shoulder_angle}$

$j3 = 180.0 - \text{elbow_angle}$

Este bloque realiza la primera conversión entre los ángulos obtenidos del movimiento humano y las articulaciones del robot.

Es importante remarcar que esta conversión es **provisional**.

No se está afirmando todavía que estas ecuaciones sean la correspondencia cinemática definitiva del robot.

Articulación J1

$j1 = \text{shoulder_angle}$

La primera articulación recibe directamente el ángulo calculado para la orientación del brazo.

Por tanto:

J1 = orientación del brazo

En esta implementación:

$\text{shoulder_angle} \rightarrow J1$

Articulación J2

$j2 = -\text{shoulder_angle}$

La segunda articulación utiliza el mismo ángulo, pero invertido.

Por tanto:

$J2 = -\text{shoulder_angle}$

El signo negativo representa una inversión de la dirección del movimiento respecto a la referencia utilizada para J1.

Esta relación es propia de esta conversión provisional y deberá revisarse cuando se defina la configuración cinemática definitiva del robot.

Articulación J3

$j3 = 180.0 - \text{elbow_angle}$

La tercera articulación se calcula a partir del ángulo del codo.

En este caso no se utiliza directamente elbow_angle , sino:

$180^\circ - \text{elbow_angle}$

Esto representa una transformación del ángulo humano a la convención angular utilizada provisionalmente para la articulación J3.

Por tanto:

Ángulo del codo



$180^\circ - \text{elbow_angle}$



J3

Aplicar límites

Una vez calculadas las articulaciones, se aplican los límites definidos anteriormente.

Limitar J1

```
j1 = clamp(  
    j1,  
    J1_MIN,  
    J1_MAX  
)
```

Aquí comprobamos que j1 se encuentre dentro del intervalo:

$$-180^\circ \leq J1 \leq 180^\circ$$

Si el valor calculado supera alguno de los extremos, clamp() lo ajustará al límite correspondiente.

Limitar J2

```
j2 = clamp(  
    j2,  
    J2_MIN,  
    J2_MAX  
)
```

Se realiza el mismo proceso para J2.

El resultado queda dentro de:

$$-180^\circ \leq J2 \leq 180^\circ$$

Limitar J3

```
j3 = clamp(
```

```
j3,
J3_MIN,
J3_MAX
)
```

Finalmente se limita J3 al mismo rango provisional:

$$-180^{\circ} \leq J3 \leq 180^{\circ}$$

De esta manera, ninguna de las tres articulaciones utilizadas en esta fase puede salir del rango configurado.

Crear configuración

```
return {
  "joints": [
    j1,
    j2,
    j3,
    0.0,
    0.0,
    0.0
  ]
}
```

Una vez calculadas y limitadas las articulaciones, se crea la configuración final del robot.

La información se almacena en un diccionario con la clave:

"joints"

que contiene una lista de seis valores.

La estructura es:

```
joints
|
├─ J1
├─ J2
├─ J3
├─ J4
└─ J5
```

└─ J6

Las tres primeras articulaciones reciben los valores calculados:

J1 → j1

J2 → j2

J3 → j3

Mientras que las tres restantes se mantienen provisionalmente en:

J4 = 0°

J5 = 0°

J6 = 0°

Esto indica que en esta fase solamente se está realizando una conversión basada en las tres primeras articulaciones.

Las restantes quedan preparadas dentro de la estructura para una futura ampliación del sistema.

Guardar JSON

```
def save_robot_motion(robot_motion):
```

Esta función se encarga de guardar la configuración calculada en un archivo JSON.

Recibe:

robot_motion

que contiene la configuración de las articulaciones.

Abrir el archivo de salida

```
with open(
```

```
    OUTPUT_FILE,
```

```
    "w",
```

```
    encoding="utf-8"
```

```
) as file:
```

El archivo se abre en modo escritura mediante:

```
"w"
```

Esto permite escribir la nueva configuración de movimiento en:

data/robot_motion.json

Guardar la configuración

```
json.dump(
```

```
    robot_motion,
```

```
file,  
    indent=4  
)
```

json.dump() convierte la estructura de Python en formato JSON y la escribe en el archivo.

El parámetro:

```
indent=4
```

hace que el archivo tenga una estructura indentada y fácil de leer.

Por ejemplo, conceptualmente el archivo tendrá una estructura similar a:

```
{  
  "joints": [  
    ...,  
    ...,  
    ...,  
    0.0,  
    0.0,  
    0.0  
  ]  
}
```

Esto permite que el resultado pueda ser consultado posteriormente por otras partes del sistema.

Función principal

```
def update_robot_motion(relative_pose):
```

Esta función proporciona el flujo completo de actualización del movimiento del robot.

Su objetivo es:

1. calcular la nueva configuración del robot;
2. guardarla en el archivo JSON;
3. devolverla para que pueda utilizarse inmediatamente.

Calcular el movimiento

```
robot_motion = calculate_robot_motion(  
    relative_pose  
)
```

Primero se llama a `calculate_robot_motion()` utilizando la pose relativa obtenida anteriormente.

El resultado se almacena en:

`robot_motion`

Este objeto contiene la configuración de las articulaciones.

Guardar el movimiento

```
save_robot_motion(  
    robot_motion  
)
```

A continuación se guarda la configuración calculada en:

`robot_motion.json`

Por tanto, el resultado no solamente queda disponible en memoria, sino que también se almacena como archivo.

Devolver el resultado

```
return robot_motion
```

Finalmente, la función devuelve la configuración calculada.

Esto permite que otra parte del sistema pueda utilizar el resultado directamente sin necesidad de volver a calcularlo.

Flujo completo del archivo

El funcionamiento completo de `Robot Motion.py` puede resumirse como:

Human Pose



Relative Pose



Vectores del brazo derecho



Ángulo del brazo



Ángulo del codo



Conversión provisional



J1, J2, J3



Aplicación de límites



Configuración de articulaciones



robot_motion.json

De esta forma, este archivo representa el paso en el que la información matemática obtenida del movimiento humano comienza a transformarse en **órdenes articulares para el robot**.

Relación con los archivos anteriores

Este archivo conecta varias de las etapas que ya hemos documentado.

La relación puede representarse como:

Calibration.py



Calibración del operador

Landmark.py



Puntos corporales

Pose.py



Organización de los landmarks

Relative Pose.py



Vectores de los segmentos

PoseMath.py



Cálculo de ángulos

Limits.py



Restricciones

Robot Mapping.py



Relación dimensional humano-robot

Robot Motion.py



Configuración de articulaciones

Cada archivo tiene una responsabilidad diferente y, juntos, comienzan a formar la arquitectura completa del sistema.

Estado actual de Robot Motion

Es importante documentar que esta implementación todavía se encuentra en una fase **provisional**.

Actualmente:

- solamente se utiliza el brazo derecho para calcular el movimiento;
- se utilizan tres articulaciones (J1, J2 y J3);
- las tres articulaciones restantes se mantienen en 0.0;
- los límites articulares son provisionales;
- la conversión entre los ángulos humanos y las articulaciones del robot es provisional;
- todavía no existe una configuración física definitiva del robot.

Por tanto, este código constituye una **primera capa funcional de transformación del movimiento humano en una configuración articular robótica**, pero no representa todavía la integración definitiva con el robot físico.

Importancia dentro del proyecto

Robot Motion.py es especialmente importante porque constituye el puente entre el **análisis del movimiento humano** y la **representación del movimiento del robot**.

Hasta Relative Pose y PoseMath, el sistema está principalmente interpretando el movimiento del operador:

Persona



Landmarks



Pose



Vectores



Ángulos

A partir de Robot Motion.py, esa información empieza a convertirse en una configuración robótica:

Ángulos humanos



Conversión



J1, J2, J3



Límites



Configuración del robot

El siguiente paso natural será que esta configuración pueda ser utilizada por la capa de comunicación o control correspondiente para producir el movimiento del robot.

Por tanto, Robot Motion.py constituye la capa encargada de transformar la información geométrica obtenida de la pose humana en una configuración provisional de las articulaciones del robot, aplicando además los límites definidos para evitar valores fuera del rango permitido.

human_pose_detection.py

```
import cv2

from ultralytics import YOLO

from pose import HumanPose
from pose_drawer import draw_pose

from relative_pose import calculate_relative_arm_pose
from robot_mapping import load_robot_mapping
from robot_motion import update_robot_motion
robot_mapping = load_robot_mapping()

# -----
# Cargar modelo
# -----

model = YOLO("yolo11n-pose.pt")

cap = cv2.VideoCapture(0)

while True:

    ret, frame = cap.read()

    if not ret:
        break

    # -----
    # Inferencia
    # -----

    results = model(frame)
```

```
# -----  
  
# Crear objeto Pose  
  
# -----  
  
pose = HumanPose()  
  
keypoints = results[0].keypoints  
  
if keypoints is not None and len(keypoints.xy) > 0:  
  
    puntos = keypoints.xy[0]  
    conf = keypoints.conf[0]  
  
    # Nariz  
    pose.nose.x = float(puntos[0][0])  
    pose.nose.y = float(puntos[0][1])  
    pose.nose.confidence = float(conf[0])  
  
    # Ojos  
    pose.left_eye.x = float(puntos[1][0])  
    pose.left_eye.y = float(puntos[1][1])  
    pose.left_eye.confidence = float(conf[1])  
  
    pose.right_eye.x = float(puntos[2][0])  
    pose.right_eye.y = float(puntos[2][1])  
    pose.right_eye.confidence = float(conf[2])  
  
    # Orejas  
    pose.left_ear.x = float(puntos[3][0])  
    pose.left_ear.y = float(puntos[3][1])
```

```
pose.left_ear.confidence = float(conf[3])
```

```
pose.right_ear.x = float(puntos[4][0])
```

```
pose.right_ear.y = float(puntos[4][1])
```

```
pose.right_ear.confidence = float(conf[4])
```

```
# Hombros
```

```
pose.left_shoulder.x = float(puntos[5][0])
```

```
pose.left_shoulder.y = float(puntos[5][1])
```

```
pose.left_shoulder.confidence = float(conf[5])
```

```
pose.right_shoulder.x = float(puntos[6][0])
```

```
pose.right_shoulder.y = float(puntos[6][1])
```

```
pose.right_shoulder.confidence = float(conf[6])
```

```
# Codos
```

```
pose.left_elbow.x = float(puntos[7][0])
```

```
pose.left_elbow.y = float(puntos[7][1])
```

```
pose.left_elbow.confidence = float(conf[7])
```

```
pose.right_elbow.x = float(puntos[8][0])
```

```
pose.right_elbow.y = float(puntos[8][1])
```

```
pose.right_elbow.confidence = float(conf[8])
```

```
# Muñecas
```

```
pose.left_wrist.x = float(puntos[9][0])
```

```
pose.left_wrist.y = float(puntos[9][1])
```

```
pose.left_wrist.confidence = float(conf[9])
```

```
pose.right_wrist.x = float(puntos[10][0])
```

```
pose.right_wrist.y = float(puntos[10][1])
```

```
pose.right_wrist.confidence = float(conf[10])
```

```
# Caderas
```

```
pose.left_hip.x = float(puntos[11][0])
```

```
pose.left_hip.y = float(puntos[11][1])
```

```
pose.left_hip.confidence = float(conf[11])
```

```
pose.right_hip.x = float(puntos[12][0])
```

```
pose.right_hip.y = float(puntos[12][1])
```

```
pose.right_hip.confidence = float(conf[12])
```

```
# Rodillas
```

```
pose.left_knee.x = float(puntos[13][0])
```

```
pose.left_knee.y = float(puntos[13][1])
```

```
pose.left_knee.confidence = float(conf[13])
```

```
pose.right_knee.x = float(puntos[14][0])
```

```
pose.right_knee.y = float(puntos[14][1])
```

```
pose.right_knee.confidence = float(conf[14])
```

```
# Tobillos
```

```
pose.left_ankle.x = float(puntos[15][0])
```

```
pose.left_ankle.y = float(puntos[15][1])
```

```
pose.left_ankle.confidence = float(conf[15])
```

```
pose.right_ankle.x = float(puntos[16][0])
```

```
pose.right_ankle.y = float(puntos[16][1])
```

```
pose.right_ankle.confidence = float(conf[16])
```

```
pose.print_pose()
```

```
relative_pose = calculate_relative_arm_pose(pose)
```

```
print(relative_pose)

robot_motion = update_robot_motion(relative_pose)

print(robot_motion)
draw_pose(frame, pose)

if cv2.waitKey(1) & 0xFF == ord("q"):
    break

cap.release()
cv2.destroyAllWindows()
```

human_pose_detection.py Explicación

Importación de librerías y módulos

```
import cv2
```

```
from ultralytics import YOLO
```

```
from pose import HumanPose
```

```
from pose_drawer import draw_pose
```

```
from relative_pose import calculate_relative_arm_pose
```

```
from robot_mapping import load_robot_mapping
```

```
from robot_motion import update_robot_motion
```

Este bloque importa todas las herramientas necesarias para ejecutar el sistema completo de detección de pose y procesamiento del movimiento.

OpenCV

```
import cv2
```

cv2 corresponde a OpenCV, la librería utilizada para trabajar con la cámara y con los frames de vídeo.

En este archivo se utiliza principalmente para:

- acceder a la cámara;
- capturar imágenes;
- controlar el ciclo de ejecución;
- detectar la pulsación de la tecla q;
- cerrar correctamente la cámara y las ventanas.

YOLO

```
from ultralytics import YOLO
```

Aquí se importa la clase YOLO de la librería Ultralytics.

En este proyecto se utiliza un modelo YOLO especializado en **detección de pose humana**.

Su función será analizar cada frame de la cámara y localizar los diferentes puntos corporales de la persona.

HumanPose

```
from pose import HumanPose
```

Se importa la clase HumanPose desarrollada anteriormente.

Esta clase permite crear una estructura propia del proyecto donde almacenar los diferentes landmarks detectados.

Por tanto, el resultado de YOLO se transforma posteriormente en un objeto:

HumanPose

que contiene los puntos corporales organizados.

`draw_pose`

```
from pose_drawer import draw_pose
```

Esta función permite representar visualmente la pose detectada sobre el frame de la cámara.

Su función es principalmente de **visualización y depuración**.

`calculate_relative_arm_pose`

```
from relative_pose import calculate_relative_arm_pose
```

Se importa la función encargada de calcular los vectores relativos de los brazos.

Esta función utiliza los landmarks almacenados en HumanPose y obtiene:

Hombro → Codo

Codo → Muñeca

para ambos brazos.

`load_robot_mapping`

```
from robot_mapping import load_robot_mapping
```

Esta función permite cargar la configuración de mapeo entre las dimensiones humanas y las del robot.

`update_robot_motion`

```
from robot_motion import update_robot_motion
```

Esta función se encarga de transformar la información de movimiento obtenida de la pose relativa en una configuración de articulaciones del robot.

De esta forma, este bloque de importaciones ya muestra cómo se conectan las diferentes capas del proyecto.

Cámara

↓

YOLO

↓

HumanPose

↓

Relative Pose



Robot Mapping



Robot Motion

Cargar el mapeo del robot

```
robot_mapping = load_robot_mapping()
```

Aquí se carga la configuración de mapeo del robot.

La función recupera las medidas de calibración del operador y calcula los factores de escala correspondientes.

El resultado se almacena en:

```
robot_mapping
```

Aunque esta variable queda preparada desde el inicio del programa, en la implementación actual la función `update_robot_motion()` no utiliza directamente este objeto.

Por tanto, en esta fase el mapeo queda **cargado y disponible**, pero la transformación utilizada posteriormente en Robot Motion se basa todavía en la conversión angular provisional definida en ese archivo.

Cargar modelo

```
model = YOLO("yolo11n-pose.pt")
```

Aquí se carga el modelo YOLO utilizado para la detección de pose humana.

El archivo:

```
yolo11n-pose.pt
```

corresponde al modelo de pose utilizado por el sistema.

La variable:

```
model
```

contiene el modelo ya preparado para realizar inferencias.

A partir de este momento, el programa puede proporcionarle una imagen y obtener como resultado información sobre los puntos corporales detectados.

Inicializar la cámara

```
cap = cv2.VideoCapture(0)
```

Aquí se inicializa la cámara utilizada como entrada del sistema.

El valor:

```
0
```

indica que se utiliza la cámara principal o primera cámara disponible para OpenCV.

El objeto:

cap

será utilizado posteriormente para obtener continuamente nuevos frames.

Por tanto:

Cámara



cap



Frames

Bucle principal

while True:

Aquí comienza el bucle principal del programa.

El sistema funcionará continuamente mientras este bucle permanezca activo.

En cada iteración se realizará aproximadamente el siguiente proceso:

Capturar frame



Ejecutar YOLO



Crear HumanPose



Extraer landmarks



Calcular pose relativa



Calcular movimiento robot



Dibujar pose



Repetir

Esto permite que el sistema procese el movimiento del operador prácticamente en tiempo real.

Capturar un frame

```
ret, frame = cap.read()
```

En cada iteración se intenta obtener un nuevo frame de la cámara.

Se obtienen dos valores:

ret

frame

frame contiene la imagen capturada.

ret indica si la captura se ha realizado correctamente.

Comprobar captura

if not ret:

break

Si la cámara no consigue proporcionar correctamente un frame, el programa abandona el bucle principal.

Esto evita continuar ejecutando el procesamiento sobre una imagen que no se ha podido obtener correctamente.

Inferencia

```
results = model(frame)
```

Aquí se produce la inferencia del modelo YOLO.

Se proporciona:

frame

como entrada al modelo.

YOLO analiza la imagen y devuelve información sobre las personas y sus puntos corporales detectados.

El resultado se almacena en:

results

A partir de aquí comienza la transformación de la información proporcionada por YOLO a las estructuras propias del proyecto.

Crear objeto Pose

```
pose = HumanPose()
```

Se crea una nueva instancia de HumanPose.

Este objeto comenzará inicialmente con sus landmarks en sus valores predeterminados.

Posteriormente, los datos obtenidos de YOLO se copiarán dentro de este objeto.

Conceptualmente:

YOLO



detecciones



HumanPose

Esto permite que el resto del proyecto trabaje con una estructura propia y no directamente con el formato interno de YOLO.

Obtener keypoints

```
keypoints = results[0].keypoints
```

Aquí se accede a los puntos corporales detectados por YOLO.

results[0] representa el resultado correspondiente a la primera detección utilizada.

La propiedad:

```
.keypoints
```

contiene los puntos corporales detectados por el modelo de pose.

Estos puntos serán posteriormente utilizados para rellenar el objeto HumanPose.

Comprobar que existen keypoints

```
if keypoints is not None and len(keypoints.xy) > 0:
```

Antes de intentar acceder a los puntos, el programa comprueba que realmente existan detecciones.

Se realizan dos comprobaciones:

```
keypoints is not None
```

comprueba que YOLO haya proporcionado información de keypoints.

Y:

```
len(keypoints.xy) > 0
```

comprueba que exista al menos una detección.

Esto evita intentar acceder a datos inexistentes cuando no se detecta ninguna persona.

Obtener coordenadas y confianza

```
puntos = keypoints.xy[0]
```

```
conf = keypoints.conf[0]
```

Aquí se extraen dos tipos de información.

Coordenadas

puntos

contiene las coordenadas x e y de los diferentes puntos corporales.

Confianza

conf

contiene el nivel de confianza asociado a cada detección.

Por tanto, para cada landmark disponemos de:

(x, y, confidence)

que coincide directamente con la estructura definida anteriormente en Landmark.

Nariz

```
pose.nose.x = float(puntos[0][0])
```

```
pose.nose.y = float(puntos[0][1])
```

```
pose.nose.confidence = float(conf[0])
```

Aquí se copia la información correspondiente a la nariz desde YOLO al objeto HumanPose.

El índice:

puntos[0]

corresponde al primer punto corporal utilizado por el modelo, la nariz.

Se almacenan:

X

Y

Confidence

dentro de:

pose.nose

Por tanto, YOLO proporciona los datos y HumanPose los organiza utilizando nuestra clase Landmark.

Ojos

```
pose.left_eye.x = float(puntos[1][0])
```

```
pose.left_eye.y = float(puntos[1][1])
```

```
pose.left_eye.confidence = float(conf[1])
```

```
pose.right_eye.x = float(puntos[2][0])
```

```
pose.right_eye.y = float(puntos[2][1])
```

```
pose.right_eye.confidence = float(conf[2])
```

Se realiza el mismo procedimiento para el ojo izquierdo y el ojo derecho.

Cada punto obtenido de YOLO se convierte en información almacenada dentro del correspondiente Landmark.

Orejas

```
pose.left_ear.x = float(puntos[3][0])
```

```
pose.left_ear.y = float(puntos[3][1])
```

```
pose.left_ear.confidence = float(conf[3])
```

```
pose.right_ear.x = float(puntos[4][0])
```

```
pose.right_ear.y = float(puntos[4][1])
```

```
pose.right_ear.confidence = float(conf[4])
```

Se almacenan las coordenadas y la confianza de las dos orejas.

Hombros

```
pose.left_shoulder.x = float(puntos[5][0])
```

```
pose.left_shoulder.y = float(puntos[5][1])
```

```
pose.left_shoulder.confidence = float(conf[5])
```

```
pose.right_shoulder.x = float(puntos[6][0])
```

```
pose.right_shoulder.y = float(puntos[6][1])
```

```
pose.right_shoulder.confidence = float(conf[6])
```

Aquí se almacenan los puntos correspondientes a los hombros.

Estos landmarks son especialmente importantes porque posteriormente se utilizarán para calcular los vectores del brazo:

Hombro → Codo

Codos

```
pose.left_elbow.x = float(puntos[7][0])
```

```
pose.left_elbow.y = float(puntos[7][1])
```

```
pose.left_elbow.confidence = float(conf[7])
```

```
pose.right_elbow.x = float(puntos[8][0])
```

```
pose.right_elbow.y = float(puntos[8][1])
```

```
pose.right_elbow.confidence = float(conf[8])
```

Aquí se almacenan los puntos de los codos.

Los codos son especialmente importantes porque conectan los dos segmentos que posteriormente serán analizados:

Hombro → Codo

Codo → Muñeca

Muñecas

```
pose.left_wrist.x = float(puntos[9][0])
```

```
pose.left_wrist.y = float(puntos[9][1])
```

```
pose.left_wrist.confidence = float(conf[9])
```

```
pose.right_wrist.x = float(puntos[10][0])
```

```
pose.right_wrist.y = float(puntos[10][1])
```

```
pose.right_wrist.confidence = float(conf[10])
```

Se almacenan las coordenadas y confianza de ambas muñecas.

Con hombro, codo y muñeca ya disponemos de los principales landmarks necesarios para el análisis de los brazos.

Caderas

```
pose.left_hip.x = float(puntos[11][0])
```

```
pose.left_hip.y = float(puntos[11][1])
```

```
pose.left_hip.confidence = float(conf[11])
```

```
pose.right_hip.x = float(puntos[12][0])
```

```
pose.right_hip.y = float(puntos[12][1])
```

```
pose.right_hip.confidence = float(conf[12])
```

Se almacenan los puntos correspondientes a las caderas.

Aunque actualmente el procesamiento del movimiento robótico se centra en los brazos, estos landmarks permiten disponer de una representación corporal más completa.

Rodillas

```
pose.left_knee.x = float(puntos[13][0])
```

```
pose.left_knee.y = float(puntos[13][1])
```



```
pose.left_knee.confidence = float(conf[13])
```

```
pose.right_knee.x = float(puntos[14][0])
```

```
pose.right_knee.y = float(puntos[14][1])
```

```
pose.right_knee.confidence = float(conf[14])
```

Se almacenan los puntos correspondientes a las dos rodillas.

Tobillos

```
pose.left_ankle.x = float(puntos[15][0])
```

```
pose.left_ankle.y = float(puntos[15][1])
```

```
pose.left_ankle.confidence = float(conf[15])
```

```
pose.right_ankle.x = float(puntos[16][0])
```

```
pose.right_ankle.y = float(puntos[16][1])
```

```
pose.right_ankle.confidence = float(conf[16])
```

Finalmente se almacenan los puntos correspondientes a los dos tobillos.

De esta manera, el objeto HumanPose contiene los diferentes puntos corporales proporcionados por el modelo.

La estructura general queda:

HumanPose

|

|— Cabeza

| |— Nose

| |— Left Eye

| |— Right Eye

| |— Left Ear

| |— Right Ear

|

|— Brazos

| |— Left Shoulder

| |— Right Shoulder

| |— Left Elbow

```

|   ├── Right Elbow
|   ├── Left Wrist
|   └── Right Wrist
|
|   ├── Caderas
|   ├── Left Hip
|   └── Right Hip
|
|   ├── Rodillas
|   ├── Left Knee
|   └── Right Knee
|
|   └── Tobillos
|       ├── Left Ankle
|       └── Right Ankle

```

Mostrar la pose

```
pose.print_pose()
```

Aquí se solicita al objeto HumanPose que muestre la información de la pose.

Esta función se utiliza principalmente para **comprobar y depurar los datos obtenidos**.

Permite verificar que los landmarks se están obteniendo y almacenando correctamente.

Calcular pose relativa

```
relative_pose = calculate_relative_arm_pose(pose)
```

Una vez construido el objeto HumanPose, se pasa a la función `calculate_relative_arm_pose()`.

Aquí se produce el siguiente paso de procesamiento:

HumanPose



Relative Pose

La función utiliza los hombros, codos y muñecas para calcular los vectores de los segmentos de los brazos.

El resultado se almacena en:

```
relative_pose
```

que contiene:

left_arm

└─ upper_arm

└─ forearm

right_arm

└─ upper_arm

└─ forearm

Mostrar pose relativa

```
print(relative_pose)
```

Se imprime la información calculada para comprobar el resultado de los vectores.

Al igual que `pose.print_pose()`, esta impresión tiene principalmente una función de **visualización y depuración durante el desarrollo**.

Calcular movimiento del robot

```
robot_motion = update_robot_motion(relative_pose)
```

Este es uno de los pasos más importantes del flujo.

La pose relativa se proporciona a `update_robot_motion()`.

Esta función:

1. obtiene los vectores necesarios;
2. calcula los ángulos;
3. realiza la conversión provisional a articulaciones;
4. aplica los límites;
5. guarda la configuración en `robot_motion.json`;
6. devuelve la configuración calculada.

Por tanto:

Relative Pose



Robot Motion



J1, J2, J3...

Mostrar movimiento del robot

```
print(robot_motion)
```

Se muestra por pantalla la configuración calculada.

Esto permite comprobar durante las pruebas qué valores están siendo generados para las articulaciones.

Dibujar la pose

```
draw_pose(frame, pose)
```

Esta función utiliza el frame original y la pose detectada para dibujar visualmente los puntos y conexiones correspondientes.

El objetivo es poder comprobar visualmente que la detección funciona correctamente.

El flujo es:

Frame original

+

HumanPose

↓

draw_pose()

↓

Frame visualizado

Esto facilita la depuración del sistema de visión artificial.

Esperar la tecla de salida

```
if cv2.waitKey(1) & 0xFF == ord("q"):
```

```
    break
```

Esta parte permite detener el programa pulsando la tecla q.

cv2.waitKey(1) espera brevemente una entrada del teclado.

Si la tecla detectada corresponde a:

```
ord("q")
```

se ejecuta:

```
break
```

y se abandona el bucle principal.

Esto proporciona una forma sencilla de detener la captura de vídeo.

Liberar la cámara

```
cap.release()
```

Una vez terminado el bucle, se libera el recurso utilizado por la cámara.

Esto es importante para que el dispositivo pueda volver a utilizarse correctamente por otros programas.

Cerrar ventanas

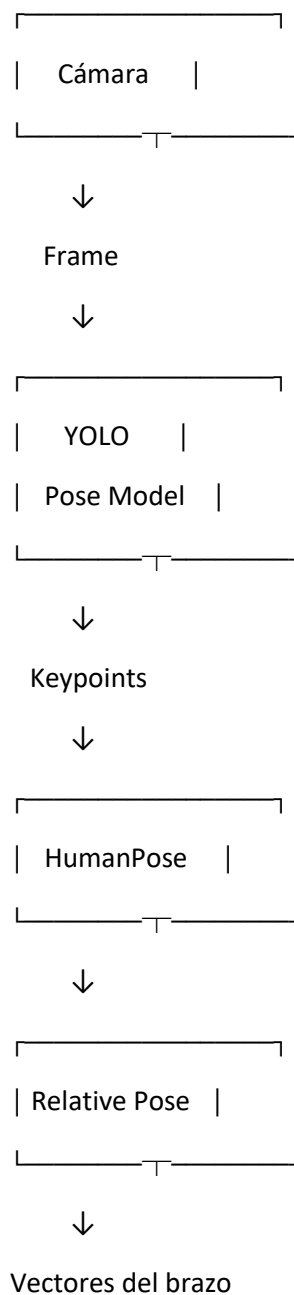
`cv2.destroyAllWindows()`

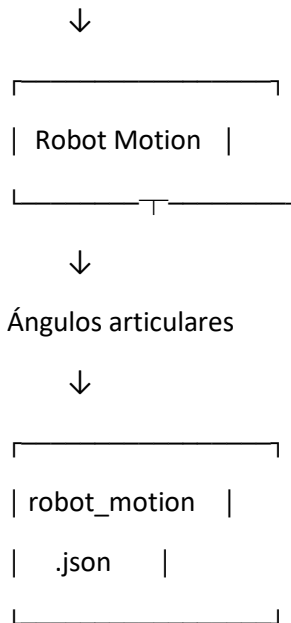
Finalmente se cierran las ventanas creadas por OpenCV.

Con esto termina correctamente la ejecución del sistema.

Flujo completo de ejecución

El funcionamiento completo de este archivo puede representarse como:





Paralelamente, la pose puede enviarse a:

HumanPose



draw_pose()



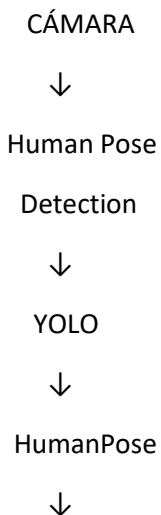
Visualización

Integración entre los diferentes módulos

Este archivo es especialmente importante porque actúa como **punto de integración de gran parte del sistema**.

Los módulos que hemos documentado anteriormente dejan de funcionar como elementos aislados y comienzan a trabajar conjuntamente.

La arquitectura puede resumirse como:



Relative Pose



PoseMath



Ángulos / vectores



Robot Motion



J1, J2, J3...

Además:

Calibration.py



calibration.json



Robot Mapping



Factores de escala

Y:

Limits.py



Restricciones



Robot Motion

De esta manera, cada módulo tiene una responsabilidad específica.

Papel de Human Pose Detection dentro del proyecto

Human Pose Detection.py representa el punto en el que la información del mundo real entra en el sistema.

Hasta este momento hemos trabajado principalmente con estructuras y cálculos:

Landmark

Pose

Vectores

Ángulos

Mapeo

En este archivo aparece finalmente la entrada real:

Persona



Cámara



Imagen



YOLO



Landmarks

A partir de esos landmarks se ejecuta el resto del procesamiento.

Por tanto, este archivo conecta el **mundo físico** con la **lógica matemática y robótica** del proyecto.

Papel de la integración general

El sistema completo puede entenderse como una cadena de transformación de información:

MOVIMIENTO HUMANO



CÁMARA



IMAGEN / FRAME



YOLO



LANDMARKS HUMANOS



HumanPose



VECTORES RELATIVOS



PoseMath



ÁNGULOS Y GEOMETRÍA



ROBOT MAPPING



ADAPTACIÓN HUMANO-ROBOT



ROBOT MOTION



ARTICULACIONES DEL ROBOT

Este flujo representa la idea fundamental de Kinema Nexus: **capturar el movimiento humano mediante visión artificial, interpretarlo matemáticamente y transformarlo en información utilizable para el control de un sistema robótico.**

Estado actual de la integración

Aunque este archivo integra prácticamente todos los módulos principales, la implementación actual sigue teniendo varios elementos provisionales.

Entre ellos:

- el modelo de YOLO utilizado;
- el uso de una única cámara;
- el cálculo basado en coordenadas 2D;
- la utilización del brazo derecho en Robot Motion;
- la conversión provisional de ángulos a J1, J2 y J3;
- los límites provisionales de las articulaciones;
- las dimensiones provisionales del robot;
- las articulaciones J4, J5 y J6 mantenidas en 0.0.

Por tanto, esta implementación representa una **integración funcional inicial del pipeline de visión y generación de movimiento**, pero todavía no constituye la configuración final del sistema robótico.

Importancia dentro de Kinema Nexus

Este archivo es el encargado de ejecutar el flujo completo de procesamiento.

Los módulos anteriores proporcionan las diferentes piezas:

Calibration

→ Medidas del operador

Landmark

→ Representación de puntos

Pose

→ Organización de la pose

PoseMath

→ Cálculos matemáticos

Relative Pose

→ Vectores relativos

Limits

→ Restricciones

Robot Mapping

→ Correspondencia dimensional

Robot Motion

→ Configuración articular

Mientras que Human Pose Detection.py **coordina estas piezas durante la ejecución**, tomando como entrada los frames de la cámara y haciendo avanzar la información por las diferentes etapas.

Por tanto, puede considerarse uno de los principales puntos de integración del proyecto.

En resumen, Human Pose Detection.py constituye el punto de entrada de los datos reales del sistema y coordina el procesamiento desde la captura de la persona mediante la cámara y su detección mediante YOLO hasta la generación de una configuración provisional de movimiento para el robot.